
TED

The Code Book

How your assistant actually works, and how to change it

Written for **Charlie Rowenhorst**

`~/ted-ai · branch arch/single-call`

August 19, 2026

40 Python files · ~24,300 lines · the window · the dashboard

Before you start

This book has a companion, and the companion is the code itself.

Every source file in `~/ted-ai` now opens with a block that looks like this:

```
# =====  
#  READING THIS FILE           The Ted Code Book – Chapter 7 (§7.1 – §7.5)  
# =====
```

and every important section inside those files is tagged like this:

```
# [BOOK §6.6] — GATE 5: the deterministic allowlist —————
```

So the two directions both work. Reading the book and want to see the real thing? Search the repo for the section number. Staring at a piece of code you do not understand? The tag tells you which chapter explains it.

```
cd ~/ted-ai  
grep -rn "BOOK §6.6" .
```

One promise about accuracy. Everything in this book was read out of the working tree on August 19, 2026, not remembered. Where a line number or a file size appears, it was true that day. Code moves; the anchors move with it, the numbers do not. Trust the anchors over the numbers.

How to read this

You do not have to read it in order, and you almost certainly should not read it cover to cover on the first pass.

If you have never opened the code before: read Part I (Chapters 1–3), then Chapter 6 — the ladder — and then stop. That is the spine of the whole program. Everything else is detail hanging off it.

If you want to change something today: go straight to Chapter 30. It walks one real change end to end, with the commands, and it assumes you have read nothing else.

If you want to add a new capability: Chapter 31. It is the three-file process, in order, with the step people forget marked as the step people forget.

If something is broken: Chapter 32 (reading an error), then Chapter 35 (the problems that are already known about — check there before you spend an afternoon rediscovering one).

Part VI is a reference. Do not read it. Look things up in it.

A note on the level

This is written on the assumption that you know what a variable, a function and a loop are, and that you have not spent years reading other people's Python. So:

- Every piece of syntax that is not obvious gets explained the first time it shows up, in Chapter 2, and again briefly at the point of use.
- Every acronym gets spelled out once. API, JSON, SQL, TTS, STT, FTS5, AEC.
- Where three sentences are needed, there are three sentences. Nothing here is compressed to look clever.

What is *not* assumed is that any of this is obvious in hindsight. Several of the decisions in Ted look strange until you know the bug they came from. Those stories are in here, because knowing why a piece of code is weird is the difference between changing it safely and reintroducing a bug that took a week to find.

Contents

PART I — ORIENTATION

1. What Ted is
2. The Python you need
3. Ted's own vocabulary

PART II — THE LIFE OF ONE MESSAGE

4. Starting up: `hud.py`
5. Getting the message in
6. The ladder: `_respond()`
7. Choosing the tool menu: `core/routing.py`
8. What Ted knows before it answers
9. Building the prompt: `core/llm.py`
10. Making the call: `core/providers.py`
11. Streaming, and tools
12. Speaking and showing the answer
13. After the turn

PART III — MEMORY

14. `core/memory.py` — the database
15. The dashboard
16. The notebook
17. The knowledge base
18. Lingo

PART IV — THE WINDOW

19. `ui/ted_hud.html` — the shape of it
20. The `tedHud` object, and the bridge
21. Adding a button, end to end

PART V — VOICE AND AUDIO

22. `core/voice.py` — ears and mouth
23. `core/audio.py` — and why interrupting is hard

PART VI — THE OTHER MODULES (reference)

- 24. Your Mac: actions, computer, system_state
- 25. Your accounts: calendar, notes, email, messages, bouncer
- 26. Music
- 27. Seeing: attachments, screen, codebase
- 28. Doing things later: assistant, proactive, routines, the daemon
- 29. Plumbing: telemetry, remote, features, paths, logs, hud_bridge, intents

PART VII — WORKING ON TED YOURSELF

- 30. Your first change, step by step
- 31. Adding a tool, end to end
- 32. How to read an error
- 33. The test suite
- 34. The rules this codebase lives by
- 35. What is already known to be wrong

APPENDICES

- A. Every file, one line each
- B. The anchor index
- C. Glossary

PART I — ORIENTATION

Chapter 1 — What Ted is

§1.1 The one-paragraph version

Ted is a Python program that runs on your Mac. It opens a window, you type in it, and it answers — but the answering happens somewhere else. Ted does not contain a language model. It contains everything *around* a language model: a window, a database of what it knows about you, a list of things it is allowed to do to your computer, and a few thousand lines deciding what to send, when to send it, and what to do with what comes back.

That is the honest shape of it. The intelligence is rented. The judgment is yours, and it is written in this repository.

§1.2 Three programs wearing one coat

When Ted is running there are actually three things going, and knowing which is which saves a lot of confusion when something breaks.

1. The Python process. Started by `python hud.py`. This is Ted. It holds the conversation, the memory, the threads, and the connection to the model. When people say "Ted crashed", this is what crashed.

2. The window. A real browser engine, rendering `ui/ted_hud.html` inside a Mac window. It is a web page. It has its own JavaScript, its own state, and it cannot see any of Python's variables. The two talk through a narrow bridge described in Chapter 20 — and *only* through that bridge.

3. A small web server. Flask, on `127.0.0.1:5175`, started on a background thread by `hud.py`. The Memory, Notebook and Diagnostics panels inside Ted's window are web pages served by this. You can also open them in an ordinary browser while Ted is running, which is often easier for reading a lot of rows.

There is a fourth thing that runs even when Ted is closed — `ted_daemon.py`, the calendar watcher — but it is deliberately tiny and independent. Chapter 28.

Why this matters in practice. If the window looks frozen but the log is still printing, the Python side is fine and the JavaScript is stuck. If the Memory panel is blank but everything else works, port 5175 is the problem, not Ted. Knowing there are three programs turns "it's broken" into a question you can actually answer.

§1.3 The folder map

```
~/ted-ai/
├─ hud.py                the starting pistol – you run this
├─ ted_daemon.py         the calendar watcher, runs on its own
├─ config.py             your keys and settings (NOT in git)
├─ config.example.py     the template for the above
├─
├─ core/                 almost all of Ted
│   ├─ app.py            THE MONOLITH. The ladder lives here.
│   ├─ llm.py            prompts, the streamed turn, memory assembly
│   ├─ providers.py      the one door every thought goes through
│   ├─ routing.py        which tools this message gets, and which brain
│   ├─ tools.py          the tool menu the model reads
│   ├─ tool_handlers.py  what several of those tools actually do
│   ├─ memory.py         the SQLite database
│   ├─ intents.py        pure text helpers, heavily tested
│   ├─ voice.py          ears and mouth
│   ├─ audio.py          raw audio, and barge-in
│   └─ ...25 more        one subject each – see Appendix A
├─
├─ dashboard/           the Flask app behind the panels
├─ ui/ted_hud.html       the entire window: layout, style, behaviour
├─ native/              Swift: the audio engine and the launcher
├─ tests/               32 files, ~1,150 checks
├─ tools/               build scripts and ted_map.py
├─ docs/                handoffs and design notes, including this book
└─ data/                memory.db, logs, runtime state (NOT in git)
```

Two folders deserve a warning.

`data/` is not in version control and holds everything Ted knows. Deleting it does not break Ted; it gives Ted amnesia. Back it up before doing anything adventurous.

`config.py` is not in version control and holds live API keys. Nothing in the codebase is allowed to read it back to you — see `core/codebase.py`, §27.3, for how that is enforced rather than merely intended.

§1.4 What Ted is not

It is worth being precise about this, because the gap between what a project *looks* like it does and what it does is where wasted afternoons live.

Ted does not think locally by default. It sends your message to Groq, a hosted service, and Groq runs the model. There *is* a genuine local fallback — a model running under Ollama on your Mac — but it is a fallback, not the norm. Chapter 10.

Ted cannot edit its own code without being asked. It can read all of it. Writing goes through the same yes/no confirmation as sending a message, and refuses to run without it. That is a decision, not a gap. §27.3.

Ted has no agents, no sub-assistants, no crew. One model, one conversation. The decision was routing, not named sub-agents, and it has held.

Ted boots muted. It is a chat application that can also speak, not a voice assistant that also has a text box. That flipped in August 2026 for a practical reason: you are at college, surrounded by people, and talking out loud at a laptop is unusable most of the time.

Chapter 2 — The Python you need

You will meet all of this within ten minutes of opening `core/app.py`. None of it is hard; all of it is easier to read once somebody has named it.

§2.1 Modules and imports

A **module** is a `.py` file. A **package** is a folder of them with an `__init__.py` inside, which is Python's way of saying "this folder is a thing you can import from".

```
from core import memory, llm          # get the whole module
from core.actions import open_app     # get one name out of a module
import re                             # a module from Python itself
```

After `from core import memory`, you write `memory.save_fact(...)`. After `from core.actions import open_app`, you write `open_app(...)` directly. Both appear constantly in Ted; which one is used is mostly about how often the module is called.

Imports inside functions. You will see this in Ted:

```
def something(self):
    from core import email as _email_mod
```

That is deliberate, not sloppy. It delays loading a module until the moment it is needed, which keeps startup fast and stops a broken optional dependency from preventing Ted from launching at all.

§2.2 Functions, classes, methods, `self`

A **function** is a named piece of work.

```
def looks_like_failure(result):
    return "couldn't" in result.lower()
```

A **class** is a template for an object that holds data and the functions that work on it. `TedApi` in `core/app.py` is the only significant class in the project.

```
class TedApi:
    def __init__(self):          # runs once, when TedApi() is created
        self.muted = True      # a piece of data that lives as long as Ted
        self.last_reply = ""

    def stop(self):             # a "method" – a function attached to the class
        self.interrupt_speech = True
```

`self` is the object itself. Python passes it to every method automatically; you never supply it when calling.

`api.stop()` runs `stop(api)`.

Anything assigned to `self.something` survives between calls. That is exactly what "Ted remembers this during the session" means, mechanically. If you want to know what Ted can hold in its head, read `TedApi.__init__` — it is the honest list.

§2.3 Dicts, lists, tuples

Ted is mostly dictionaries, so this one is worth getting comfortable with.

```
args = {"name": "Spotify", "browser": "Brave"}  # a dict: keys -> values
args["name"]                                  # "Spotify" – crashes if absent
args.get("browser", "")                       # "Brave" – "" if absent
```

Always use `.get()` for tool arguments. The model supplies a dictionary and anything not marked `required` in the schema may simply not be there. `args["x"]` on a missing key raises `KeyError` and kills the turn; `args.get("x", "")` gives you an empty string and lets you handle it.

```
schemas = [schema_a, schema_b]              # a list – ordered, changeable
point = (70, 320)                             # a tuple – ordered, fixed
```

Tuples are used in Ted for things that should not be edited: pairs of `(pattern, tool_names)` in the routing tables, frozen sets of tool names.

§2.4 Strings

f-strings put expressions inside text:

```
print(f"[news] {len(found)} new items")
```

Anything in `{}` is evaluated and dropped in. The `f` before the quote is what switches it on.

Triple-quoted strings span lines and are used for two different jobs:

```
def capture():
    """Record a turn and transcribe it."""    # a docstring – documentation
```

```
SCHEMA = """
CREATE TABLE IF NOT EXISTS facts (...)
"""
# just a long string
```

A **docstring** is the first thing inside a function, class or file. Python stores it, tools read it, and in Ted it is the first place to look when you want to know what something is for.

Adjacent strings join automatically, which is how the long tool descriptions are wrapped:

```
"description": (
    "Open an application on the Mac. "
    "For a website use browse_to instead."
)
```

That is one string, not two.

§2.5 `try / except`, and Ted's rule about it

```
try:
    risky_thing()
except Exception as e:
    print(f"that failed: {e}")
```

Ted uses this heavily, and there is a rule about *how*, which comes from the most expensive bug in the project's history.

Fact extraction was completely dead for five weeks and nobody knew. It asked the model for JSON, got prose back, the JSON parser raised an exception — and the exception was caught and printed to a terminal nobody was reading. The facts table had one row in it for over a month.

So the rule is: **catching an error is fine; hiding it is not.**

```
except Exception as e:
    error_log.error(f"news_watch_loop: {e}")    # goes to ted_errors.log
```

Bare `except: pass` appears in Ted only for genuine cleanup — closing an audio device on the way out — where failing really is not worth crashing over. If you add one anywhere on the reply path, you are recreating the five-week bug. §34.

§2.6 Threads

A **thread** runs at the same time as everything else.

```
threading.Thread(target=self.reminder_watch, daemon=True).start()
```

`target` is the function to run. `daemon=True` means "do not keep the program alive just for this" — when Ted wants to quit, daemon threads are killed rather than waited for. Every background watcher in Ted is a daemon thread.

Joining with a timeout is how Ted waits for something without waiting forever:

```
t.join(timeout=2.0)      # give it two seconds, then carry on regardless
```

There is a real bug buried in that line and it is worth knowing. Four memory lookups each had `join(timeout=4.0)`. Four seconds *each* is sixteen seconds total, and the reply had not even been requested yet. The fix was one shared deadline for all of them (§8.2). When you see `join`, check whose budget it is spending.

A **lock** stops two threads doing the same thing at once:

```
with self._shutdown_lock:
    ...
```

Ted's `busy` lock has its own cautionary tale — `ask()` used to take it before doing anything, which queued "stop" behind the thing it was meant to stop. §5.1.

§2.7 Generators and `yield`

This is the one piece of syntax that genuinely changes how you read `core/llm.py`, so it gets a longer explanation.

An ordinary function runs once and returns once:

```
def get_reply():
    return "the whole answer, eventually"
```

A function containing `yield` behaves differently. It runs until the first `yield`, hands you that value, and **pauses** — with all of its variables still alive. Ask for the next value and it picks up exactly where it stopped.

```
def ask_streaming(...):
    for chunk in response:
        yield chunk.text      # hand back a piece, then wait
```

The caller loops over it:

```
for piece in ask_streaming(...):
    show(piece)                # display each piece the instant it arrives
```

That is how streaming works. Ted starts talking after the first few words rather than after the last, and the code that produces the words and the code that speaks them are still two separate, readable functions.

§2.8 Decorators

A line starting with `@` above a function changes what that function is.

```
@property
def muted(self):
    return self._muted
```

`@property` makes a method look like a plain attribute from outside. You write `api.muted` and Python quietly calls the function. Ted uses it where reading or writing needs to also *do* something — setting `api.muted = True` also switches the microphone off, in one place, so the two can never disagree.

```
@dataclass
class ReflexPlan:
    calls: list
```

`@dataclass` writes `__init__` for you from a list of fields. It is shorthand for "a class that is mostly a bag of named values".

```
@app.route("/api/chats")
def list_chats():
    ...
```

That is Flask's. It means "when a browser asks for this URL, run this function". Chapter 15.

§2.9 Regular expressions

A **regular expression** — regex — is a pattern for matching text. Ted uses them a lot in `core/intents.py` and `core/routing.py`.

You do not need to be able to write these to work on Ted. You do need to read them well enough to add a word to a list, which is most of what you will actually do.

```
_STOP_RE = re.compile(r"\b(?:stop|quit|cancel)\b", re.I)
```

Piece	Means
<code>r"..."</code>	raw string — backslashes stay literal. Always use it for regex.
<code>\b</code>	word boundary. <code>\bstop\b</code> matches "stop" but not "stopwatch".
<code>(?: ...)</code>	a group, not captured. Just for holding an <code>\ </code> together.
<code>\ </code>	or
<code>.</code>	any one character
<code>.*</code>	any number of any characters
<code>?</code>	the thing before it is optional
<code>+</code>	one or more
<code>^ \$</code>	start / end of the text
<code>re.I</code>	ignore case

So `r"\b(?:stop|quit|cancel)\b"` with `re.I` means: the whole word stop, quit or cancel, in any capitalisation, anywhere in the text.

Adding your own wording to a list is usually the entire fix when Ted does not respond to how you actually say something.

§2.10 Odds and ends you will hit

The walrus, `:=` — assign and test in one step.

```
if (revised := _revised_message_args(text, args)):
```

means "work out `revised` ; if it is truthy, enter the `if` , and use it inside".

`getattr(obj, "name", default)` — read an attribute that might not exist, without crashing.

```
was_speaking = getattr(engine, "_playing", False)
```

`**kwargs` — collect every keyword argument into a dictionary, so a function can forward whatever it was given without listing every option.

```
def chat_create(**kwargs):
    return _groq.chat.completions.create(**kwargs)
```

Type hints — `def active_provider() -> str:` — are documentation. Python does not enforce them and they change nothing at runtime.

A one-element list used as a box:

```
_USAGE_SUPPORTED = [True]
```

Reassigning a module-level variable from inside a function needs a `global` declaration; *mutating* a list does not. So a one-element list is used as a little mutable box, and the flag is `_USAGE_SUPPORTED[0]`. It is a trick, and now you know it is a trick.

Chapter 3 — Ted's own vocabulary

These words appear throughout the code and throughout this book. They are not standard Python terms; they are this project's terms.

Word	What it means here
the ladder	<code>TedApi._respond()</code> in <code>core/app.py</code> . The series of cheap checks every message walks down. Chapter 6.
a rung	One of those checks. A rung either handles the message and returns, or passes it down.
gate 5	What is left of the old regex dispatch — a short allowlist of message shapes handled without a model. §6.6.
a reflex	A complete, reversible app open/close handled with zero tokens and no model call. §7.4.
a routine	A phrase you defined yourself that runs a fixed list of actions. Also zero tokens. §28.3.
the tool menu	The list of tool descriptions sent with a message. Not the whole catalogue — a subset chosen per turn. §7.2.
a schema	One tool's description: name, when to use it, what arguments it takes. §11.4.
the door	<code>chat_create()</code> in <code>core/providers.py</code> . Every thought leaves through it. §10.1.
the handover	Groq failing and the identical request being retried on the local Ollama model. §10.4.
the pin	The header dropdown forcing cloud or local. A pin is obeyed, not preferred. §10.6.
the HUD	The window. <code>ui/ted_hud.html</code> . Short for heads-up display, a name from the voice-orb era.
the bridge	The two narrow channels between Python and the window. Chapter 20.
the monolith	<code>core/app.py</code> . Nearly 4,000 lines. Everyone knows. §35.
the honesty rule	Action tools report what actually happened, and Ted says that verbatim. §11.8.
barge-in	Interrupting Ted by talking over it. §23.
AEC	Acoustic echo cancellation — stopping Ted hearing itself through the speakers.
STT / TTS	Speech to text (Whisper) / text to speech (Kokoro).
a fact	One stored statement about you, injected into every prompt. §14.2.
a session memory	A short first-person memory of a whole conversation. Most conversations get none, on purpose. §13.3.
the index line	The notebook's page names, in every prompt. Contents are never in it. §16.3.
Cowork vs Claude Code	Two different tools you use on this repo. Cowork runs in a Linux sandbox — it can read and edit files but cannot run Ted, touch audio, or call macOS. Claude Code runs on the Mac and can. §34.

PART II — THE LIFE OF ONE MESSAGE

This is the part of the book worth reading properly. Everything else in Ted hangs off the path described in these ten chapters. If you understand what happens between pressing Enter and Ted answering, you can change almost anything.

Chapter 4 — Starting up: `hud.py`

File: `hud.py`, 345 lines. **Anchor:** [BOOK §4]

§4.1 What runs when you type `python hud.py`

Python reads the file from top to bottom, executing as it goes. There is no "main function" that gets called; the file is the program, and the `if __name__ == "__main__":` block at the bottom is where the interesting part starts.

In order:

1. **Signal setup for the Dock launcher.** macOS plumbing. Skip it on a first read — it is genuinely unrelated to how Ted thinks. (It exists because Python only runs signal handlers when the main thread is executing Python bytecode, and `webview.start()` hands the main thread to macOS for the rest of the session. So a Dock click was being silently dropped. The fix is a dedicated thread blocking on `sigwait`.)
2. **The `sys.path` line.** `python sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))`
This makes `from core.app import TedApi` work no matter which folder you were in when you ran the command. Without it, running Ted from anywhere but `~/ted-ai` fails with an import error.
3. **`api = TedApi()`** — this one line builds the entire assistant. Every piece of state Ted holds is created by `TedApi.__init__`. Nothing is running yet; the object simply exists.
4. **The dashboard thread.**
5. **The window**, and `webview.start()`.

§4.2 The window, and the bridge

```
window = webview.create_window(  
    f"Ted {TED_VERSION}",  
    UI_HTML,                # ui/ted_hud.html  
    js_api=api,             # <-- this is the whole bridge  
    width=1100, height=720,  
    background_color="#171614",  
    text_select=True,  
)  
api.window = window
```

`js_api=api` is the most important argument in the file. It hands the `TedApi` object to the browser engine and says: JavaScript in this page may call any public method on this object.

So when the page runs

```
window.pywebview.api.ask("what's the weather")
```

it is calling `TedApi.ask("what's the weather")` in Python. That is the JavaScript-to-Python half of the bridge, and it is the only one.

`api.window = window` closes the loop in the other direction: now Python holds a reference to the window and can run JavaScript in it. That is the Python-to-JavaScript half, and it lives in `core/hud_bridge.py` . §20.3.

Note what this design *rules out*. The two sides share no variables. Python cannot read a text box; JavaScript cannot read `self.muted` . Everything crosses through those two channels, which is restrictive and is also why the boundary stays comprehensible.

§4.3 The dashboard thread

```
threading.Thread(target=_start_memory_dashboard, daemon=True,
                 name="memory-dashboard").start()
```

The Memory, Notebook and Diagnostics panels are web pages served from `127.0.0.1:5175` by a Flask app. Starting it on a daemon thread means the panels always have something to load, and closing Ted kills it.

There is a defensive branch worth knowing about. If port 5175 is already taken, Ted asks whatever is holding it for `/api/version` and checks whether it is a *current* dashboard. If an old one is holding the port, chat history would silently fail to save — so Ted prints a loud warning instead of pretending everything is fine.

That is the "silent failures are the expensive ones" rule applied to a startup condition. §34.

§4.4 Teardown, and the bug that shaped it

```
def _teardown(reason):
    ...
```

This function writes the session memory, closes the database, and stops the audio engine. It is registered **three separate times**:

```
window.events.closing += lambda: _teardown("window closed")
signal.signal(signal.SIGINT, _shutdown)      # Ctrl-C
signal.signal(signal.SIGTERM, _shutdown)     # kill
atexit.register(_teardown, "process exit")   # the backstop
```

Three registrations that can race each other, so the function is **idempotent** — it uses a lock and a `_shutdown_done` flag to guarantee it only really runs once. Calling it twice is harmless.

Why three. The `session_summaries` table sat completely empty for months. The only write paths were a 30-minute idle timer and a signal handler — and closing the window fires *neither*. `pywebview` simply returns from `start()` and the process exits normally. Every exit route now lands in `_teardown`.

There is also a deadline inside it:

```
saver.join(timeout=15.0)
if saver.is_alive():
    print("[shutdown] session summary timed out; chat turns are already saved")
```

Writing a session memory needs a model call, and a provider call once ignored its nominal timeout and held the process in "saving state" indefinitely. Chat turns are persisted individually as they happen, so the summary gets a firm fifteen seconds and then Ted quits anyway.

That is a good pattern to copy: give optional work a deadline, and say out loud when the deadline is hit.

Chapter 5 — Getting the message in

There are two ways a message reaches Ted, and they converge almost immediately.

§5.1 Typed — `ask()`

Anchor: [BOOK §5.1] in `core/app.py`.

You press send. The JavaScript calls `window.pywebview.api.ask(text)`. Python's `ask` takes the busy lock, calls `_respond(text)`, and releases it.

That is the whole path. It is the shortest thing in the program.

The historical note in the code is worth internalising, because it is a good example of a bug that is invisible from the outside and obvious once named. `ask()` used to take the busy lock *before doing anything*, with an eight-second timeout. Which meant: while a turn was running, a "stop" message went to `ask()`, waited for the lock held by the turn it was trying to stop, timed out, and answered "the previous request is still finishing".

A real log shows a turn hung for forty-one seconds while three separate stop attempts were each politely refused. From the text box there was no way out. That code had been there since `ask()` was first written, which is why the symptom pre-dated every change made the week it was found.

Stop now bypasses the lock.

§5.2 Spoken — `conversation_loop()`

Anchor: [BOOK §5.2] in `core/app.py`.

A loop on a background thread that runs for the whole life of the program:

```
listen → capture audio → transcribe → strip the wake phrase → _respond → repeat
```

The listening and transcribing happen in `core/voice.py` (`capture()`, §22.3) and `core/audio.py` (the engine, Chapter 23). By the time the text reaches `_respond` it is an ordinary string and nothing downstream knows or cares that it was spoken.

Ted boots muted, so this loop does nothing until you press the mic button. That is the chat-first pivot in one line of behaviour.

Wake phrase stripping matters more than it looks. If you say "Hey Ted, what time is it", the words "Hey Ted" are removed before `_respond` sees the message. Otherwise every routing regex and every prompt would carry a greeting that means nothing. `_strip_wake_phrase` in `core/intents.py` does it, and it is forgiving about how Whisper actually transcribes your name — "Hey Tad", "Hated...", "So Ted,..." all count.

§5.3 Where they meet

Both paths land in `TedApi._respond(text)`.

This is worth stating plainly because it has a practical consequence: **a bug in one shows up in the other**. If Ted mishandles something you typed, it will mishandle the same thing spoken. There is no separate voice logic below this point, and adding some would be a mistake.

There is a third entrance — `core/remote.py`, the HTTP endpoint for iPhone Shortcuts — and it also calls `_respond`. Same rule.

Chapter 6 — The ladder

File: `core/app.py` , `TedApi._respond()` . **Anchor:** [BOOK §6]

This is the single most important function in Ted. Read it top to bottom at least once.

§6.1 Why a ladder

A message arrives. Ted could send everything to the model and let it decide. That is simpler, and it is wrong, for four separate reasons:

Speed. "Stop" must interrupt Ted *now*, not after a network round trip.

Cost. Every message to the model is charged against a per-minute token ceiling. Turning on the microphone should not spend budget.

Correctness. A language model doing "8 percent of 250" fails *silently*. A wrong number reads exactly like a right one. There is nothing to log and nothing to notice. Arithmetic belongs in Python.

State. If Ted asked you "which John?" last turn, your "the second one" is an answer, not a new request. Sending it to the model as a fresh message throws away the question.

So: a run of cheap local checks, each of which either handles the message and **returns**, or lets it fall through. Only what survives reaches the model.

The principle is *cheap gates before expensive ones*, and it is sound. What was once wrong with it was how many rungs were hardcoded — about fifty regular expressions trying to be the assistant. That was gutted deliberately. §6.6.

§6.2 Rungs 1–4: the instant ones

Rung 1 — mute / unmute. [BOOK §6.2]

Handled instantly, both directions, regardless of current state. There is a subtlety in the comment worth reading: Ted now *boots muted*, so "mute yourself" arrives while already muted far more often than it used to. An older version guarded this with `if not self.muted` , so the message fell all the way through to the model, which then cheerfully discussed the concept of muting instead of answering. Answer the intent, not the state.

Muting is also **silent on purpose**. Speaking a confirmation would be the last thing you hear after asking for quiet.

Rung 2 — stop.

Cuts Ted off. And if Ted was *not* talking, it pauses Spotify instead — because "stop" while music is playing and Ted is silent almost certainly means the music.

That inference is a small thing and it is the kind of small thing that makes an assistant feel like it is paying attention.

Rung 3 — cancel.

Stop, plus clear any pending question. That second half was a bug fix: "nevermind" during a contact disambiguation used to go silent here and leave the old question armed until it expired, so your *next* unrelated message got interpreted as an answer to it.

Rung 4 — UI commands.

"Open the chat log", "repeat that", "speak faster". These drive the window. They are not thoughts and must never reach the model.

§6.3 Rung 5: pending flows

If Ted asked you something last turn, your reply belongs to that question.

Ted holds four kinds of pending state, each as an instance variable:

Variable	The question it is holding
<code>_pending_tool_confirmation</code>	"Send this? yes/no"
<code>_pending_compose</code>	"What should the message say?"
<code>_pending_msg</code>	"Which John?"
<code>_pending_lingo</code>	"What does that word mean?"

Each has a matching resolver method. This is Ted's only real conversational state machine and it is **ad hoc**: each flow is a hand-written pair of an instance variable and a resolver. Adding a fifth means writing both by hand and remembering to clear it in the cancel branch. It works; it does not scale elegantly. Worth knowing before you add one.

Confirmations expire. A "yes" arriving ten minutes later gets "That confirmation expired, so I didn't do it" rather than sending something you have forgotten about.

§6.4 Rung 6: lingo

"When I say *the dispatch app* I mean the crew scheduling project."

Definitions are cheap, explicit, and — crucially — should be available to the *very next routing decision*, not after a background fact extraction eventually catches up. So they are handled here, immediately, and stored in their own table. Chapter 18.

Just below this rung, every message is run through `lingo.expand()`, producing `routing_text` — your message with your shorthand replaced by what it means. **That expanded version is what the routing rules below see.** Your original text is still what goes to the model.

§6.5 Rungs 7–9: the zero-token lanes

Three checks in a row, each of which can answer without any model call at all.

Routines (`routing.match_routine`). Phrases you authored yourself in the dashboard — "movie mode" closes these apps and sets that volume. Explicitly written by you, containing only low-risk actions, so spending tokens asking a model what they mean would be absurd. §28.3.

Documents (`routing.plan_document`). Only complete, unambiguous document requests qualify. Anything partial declines the whole turn.

Reflexes (`routing.plan_reflex`). "Open Spotify." Complete, reversible app open/close requests. §7.4.

All three follow the same rule: **if anything is unclear, return `None` and let the model have it**. A reflex that fires on an ambiguous request is the old regex ladder coming back.

All three also log a telemetry row even though they cost nothing, so the diagnostics panel does not make it look like Ted did less work that minute.

§6.6 Rung 10: gate 5

Anchor: `[BOOK §6.6]`, `_use_deterministic_command()`.

The short allowlist of message shapes handled in plain Python:

- voice shortcuts from `shortcuts.json`
- timers, reminders, corrections, cancellations
- explicit memory edits — "remember that...", "what do you know about me"
- microphone recalibration
- **arithmetic**

That is it. Everything the old regexes used to steal — apps, screen, calendar, notes, web, computer control — now reaches the model.

Arithmetic is the one that looks out of place and is not. It is there for the silent-failure reason in §6.1, and it is the clearest example in the codebase of the standing principle: **math in Python, words in the model**.

Returning `True` here does not answer the message. It means `_assistant_command` gets first look at it — roughly 800 lines of old dispatch that is mostly unreachable now.

§6.7 Rung 11: the model

Anchor: `[BOOK §6.7]`

Everything above was free. From here the turn costs tokens.

```
_selected_schemas = routing.select_tool_schemas(routing_text, _op_context)
_runtime = llm.ToolRuntime(schemas=_selected_schemas, dispatch=_selected_dispatch, ...)
gen = llm.ask_streaming(text, self.active_conversation, ..., tool_runtime=_runtime)
full, barged = speak_streaming(w, gen, self, speed=..., volume=...)
```

Four steps:

1. `routing.select_tool_schemas` picks a small tool menu for this message rather than the whole catalogue. Chapter 7.
2. `llm.ToolRuntime` bundles those schemas with the function that runs one when the model picks it. §11.2.
3. `llm.ask_streaming` builds the prompt, makes one streamed call, and hands back chunks. Chapters 8–11.
4. `speak_streaming` says and displays each chunk as it arrives. §12.1.

Note `_selected_dispatch`, defined inline just above. It intercepts one special tool name — `find_tools` — which is how the model asks for capabilities the router did not give it. That escape hatch is what allows the router to be approximate. §7.3.

The **empty-stream branch** at the bottom is worth reading:

```
if full.strip():
    ...
else:
    err = self._explain_empty_turn()
```

An empty stream is a *runtime failure*, not a failure to understand you. The old wording — "That request stopped before I could complete it. Nothing was changed." — was wrong in the case that mattered most: a `send_message` turn arms a confirmation and returns, so if the stream ended empty around it, you were told nothing was changed while Ted was holding a message waiting for "yes". That is the cheerful-lie failure pointed the other way. `_explain_empty_turn` now reports the pending confirmation if there is one, and names the brain that failed if one did.

§6.8 Adding your own rung

Decide first whether you actually want a rung. **You probably want a tool.**

- A **rung** is right when the message must never reach the model: it is latency-critical, it drives the window, it is conversational state, or a model would get it silently wrong.
- A **tool** is right when Ted should be able to *do a new thing*. Chapter 31.

If it really is a rung, put it in the right place. Order is meaning here: higher means "checked sooner and beats everything below it".

Then, before your `return`, do these three things. Forgetting one is the most common bug when adding a rung, and each produces a different confusing symptom:

```
engine.reset_barge_in()      # or the tail of your own voice counts as
                              # interrupting the reply to it
self.last_reply = reply      # or "repeat that" says the wrong thing
add_message(w, "ted", reply) # or the window shows nothing at all
```

And if the rung talks, `speak(w, reply, self)` after those.

Then add a test in `tests/test_pipeline.py`, which pins the interception order. If your rung breaks an existing one, that file will tell you.

Chapter 7 — Choosing the tool menu

File: `core/routing.py`, 743 lines. **Anchor:** [\[BOOK §7.2\]](#), [\[BOOK §7.4\]](#), [\[BOOK §7.5\]](#)

§7.1 Why not just send all the tools

Ted has about sixty tools. Every tool's schema — its name, its description, its argument list — is real text sent with every message that carries it. The whole catalogue was once measured at roughly **3,645 tokens against a 6,000-tokens-per-minute free tier**.

Do the arithmetic. Sending every tool on every message leaves barely enough room to say hello, and one message per minute is not a conversation.

So this file makes three cheap guesses before the model is ever asked:

1. Which small handful of tools might this need?
2. Is this so simple that no model is needed at all?
3. Cloud brain or local brain?

It guesses with regular expressions, which are dumb and instant.

§7.2 `select_tool_schemas` and `_FAMILIES`

Anchor: [\[BOOK §7.2\]](#)

`_FAMILIES` is a list of pairs: a pattern, and the tools it suggests.

```
_FAMILIES = (  
    (r"\b(?:open|close|quit|launch|start|bring up|pull up|app|window)\b",  
     ("open_app", "close_app")),  
    (r"\b(?:website|browser|browse|url|\.com\b|youtube|video|watch)\b",  
     ("play_youtube", "browse_to", "open_app")),  
    ...  
)
```

Read that as: *if the message mentions opening or launching something, they probably want the app tools*. These are **capability hints, not command parsers**. They do not decide what happens; they decide what is on the menu.

This function is allowed to be wrong, and the two ways it can be wrong are not equally bad.

Mistake	Cost
Too many tools offered	A few wasted tokens. Invisible.
Too few tools offered	The model calls <code>find_tools</code> , gets more, carries on. One extra round trip.

Only the second is even noticeable, and it self-corrects. That asymmetry is what lets this file stay simple and approximate. A router that had to be right would have to be as smart as the thing it is routing for, which defeats the purpose.

There is a real comment in the file worth quoting, because it is what going wrong actually looks like:

"play" itself was missing here, so "play a different one" arrived with an empty menu, burned a `find_tools` round trip, hit the rate limit and fell back to local.

One missing word in one regex, and the visible symptom was Ted being slow and using the wrong brain. That is the shape of bugs in this file.

If Ted never offers a tool you want, add your wording to the matching `_FAMILIES` row. That is usually the whole fix.

§7.3 `find_tools` — the escape hatch

`FIND_TOOLS_SCHEMA` at the top of the file is always in the menu, no matter what the router picked. Its description tells the model: if the tools you can see do not cover this, describe what is missing and call me.

When the model calls it, `_selected_dispatch` in `core/app.py` intercepts the name before it reaches the normal switchboard, runs `discover_tool_schemas`, and **adds the results to the live menu mid-turn** via `ToolRuntime.add_schemas`. Then it tells the model, in words: "Loaded capabilities: X, Y. Now use the appropriate tool."

This is the single design decision that makes the whole routing approach defensible. Without it, a phrasing nobody anticipated is locked out of a capability permanently. With it, the worst case is one extra round trip.

§7.4 `plan_reflex` — the zero-token lane

Anchor: [BOOK §7.4]

"Open Spotify" does not need a language model.

This function recognises **complete, reversible** app open/close requests and returns a plan that `core/app.py` executes directly. No prompt, no tokens, no network.

The word doing all the work is **complete**. Look at what it refuses:

```
_DEPENDENCY_WORDS = re.compile(r"\b(?:then|after|before|if|unless|while)\b", re.I)
```

"Open Spotify then play something" is not a reflex — there is a dependency, and handling half of it in a regex means handling the other half badly. It returns `None` and the whole turn goes to the model.

Same for an app name it cannot resolve confidently against `APPS` in `core/actions.py`. Partial match, no reflex.

A reflex that fires on an ambiguous request is the old regex ladder coming back. That is not a stylistic objection: the old ladder is what made Ted feel like "a robot spitting back answers", and removing it is the single change that most improved how Ted feels to use. §34.

§7.5 `classify_brain` — cloud or local

Anchor: `[BOOK §7.5]`

Which model answers this turn.

Rules first, because rules are instant. A message that clearly needs deep reasoning goes cloud; obvious small talk and simple lookups can go local. Only a genuinely ambiguous message pays about a tenth of a second to ask a very small local router model — that is `classify_brain_with_model`, and it exists because "is this hard?" is itself a judgment call.

The bias is deliberately toward the cloud, and the reasoning is worth copying into other decisions you make:

- Getting LOCAL wrong costs answer quality. You notice, and you cannot undo it.
- Getting CLOUD wrong costs tokens, which refill every minute.

Asymmetric mistakes deserve an asymmetric default. If you want more aggressive saving, the thresholds are in this function.

The verdict is advisory. `providers.chat_create` will still escalate to the cloud if the local brain is not actually available. Routing suggests; the provider decides what is possible. Two layers, one of which knows about availability and one of which does not — and only one of them gets to be wrong about it.

§7.6 `memory_scope_for` — how much retrieval this turn earns

Small function, real consequences.

It returns one of three scopes, which Chapter 8 uses to decide how many memory lookups to run:

Scope	What loads
<code>full</code>	facts, notebook index, past exchanges, knowledge base, session memories
<code>relevant</code>	facts, notebook index, past exchanges, knowledge base
(operational)	facts and notebook index only

An operational turn — "open Spotify" — does not need to know what you were stuck on last Tuesday. Skipping that retrieval is real latency saved on exactly the turns where latency is most noticeable.

Note what *never* gets scoped out: **facts and the notebook index, on every turn including operational ones.** That is a deliberate reversal of an earlier decision. Facts are one local database read, capped at 1,200 characters downstream, and they are exactly what makes an action honour a standing preference. "Open YouTube in Brave from now on" is stored as a fact, and the turn that needs it is an *action* turn. Scoping facts out re-broke that once already.

Chapter 8 — What Ted knows before it answers

File: `core/llm.py`, inside `ask_streaming`. **Anchor:** [BOOK §8, §9, §11.1-§11.3]

§8.1 Five lookups, at the same time

Before the model is asked anything, Ted gathers context:

Loader	What it fetches	From
<code>_load_facts</code>	stored statements about you	<code>facts</code> table
<code>_load_notebook</code>	page names and counts, never contents	<code>notebook_pages</code>
<code>_load_mem</code>	past exchanges matching this message	<code>exchanges</code> + FTS5
<code>_load_know</code>	relevant chunks of documents you filed	ChromaDB
<code>_load_sessions</code>	memories of previous conversations	<code>session_summaries</code>

Which ones run is decided by the scope from §7.6.

They run **concurrently**, each on its own thread:

```
_lk_threads = [threading.Thread(target=f, daemon=True) for f in loaders]
for _t in _lk_threads: _t.start()
```

Sequentially these would add up. In parallel the turn waits for the slowest one, not the sum.

Each loader writes its result into a shared dictionary `_ctx`, which is safe here because each writes a different key and nothing reads until they are done.

§8.2 One deadline, not five

This is the part worth remembering.

```
_ctx_deadline = _ctx_t0 + CONTEXT_BUDGET          # CONTEXT_BUDGET = 4.0
for _t in _lk_threads:
    _t.join(timeout=max(0.0, _ctx_deadline - time.time()))
```

Each `join` gets **whatever is left of the shared four seconds**, not four seconds of its own.

The previous version was `join(timeout=4.0)` per thread. Four threads, four seconds each, sixteen seconds worst case — and the model had not even been asked anything yet. Meanwhile a comment two lines above claimed a four-second budget. Two places disagreeing about one fact. §34.

The principle underneath: **retrieval is optional context; the answer is not**. Anything that has not arrived by the deadline is simply left out. Ted answers with less rather than later.

And when the budget is exceeded, it says which source was still missing:

```
print(f"[timing] context {_ctx_ms}ms" + (f" (empty: {'', '.join(_slow)}))" ...))
```

Silent latency is the expensive kind. Naming the slow source makes it findable instead of merely felt. In practice the knowledge base was the usual offender, which is why it now warms at startup instead of on the first message.

§8.3 The caps

```
def _cap(s, n):
    s = (s or "").strip()
    return s if len(s) <= n else s[:n].rsplit(" ", 1)[0] + "..."
```

Every retrieved block is truncated to a fixed number of characters:

Block	Cap
operational context	2400
known facts	1200
notebook index	600
past exchanges	1200
knowledge base	1500
past sessions	1200

Without these, the context block grows as your database fills — more facts, longer recalled exchanges — and **every single turn pays to reprocess it**. That is the slow creep that appears after a database has been in use for a while, and it is very hard to notice because it happens gradually.

Truncation is cheap insurance. The `rsplit(" ", 1)[0]` is a nicety: cut at a word boundary rather than mid-word.

§8.4 What the context block looks like

The pieces are assembled into one string:

```
(Context: Today is August 19, 2026. Known facts about Charlie: ... .
Notebook pages: fixes (4), classes (11). Use notebook_read before answering
from any of them. Relevant past exchanges: ... . CURRENT MODE: CHAT – text in
a window, mic off. ...)
```

It is sent as **one system message**, not merged into the persona. That is a performance decision, and it is the subject of §9.4.

Chapter 9 — Building the prompt

File: `core/llm.py` . **Anchor:** [BOOK §9.1] , [BOOK §9.5]

§9.1 `SYSTEM_PROMPT` — the persona

Anchor: [BOOK §9.1]

This string is Ted's personality. There is no other file, no settings screen, no fine-tuning. Everything you think of as "how Ted talks" is written here in plain English and sent at the top of every message.

Two consequences follow, and both are practical:

Changing Ted's behaviour usually means editing prose, not code. Too chatty? Too formal? Keeps restating your question? The fix is a sentence in here.

This text is charged for on every turn. It is the biggest single fixed block in the prompt. It was trimmed in August from about 1,120 tokens to about 470 — and the comment above it is precise about why that mattered:

The static prefix is billed against the tokens-per-minute limit on every single request whether or not the provider serves it from its prefix cache — caching is a latency win, not a rate-limit one.

That distinction is easy to get wrong. Caching makes a repeated prefix *fast*. It does not make it *free*. Cutting words here is worth real money; padding it is a real cost.

What was cut was not behaviour — it was the same rule stated three ways, plus the formatting rules, which moved into the per-turn mode line so a chat turn no longer pays for the voice rules and vice versa.

Historical note, settled. Fine-tuning a model on 143 hand-written examples to give Ted a personality was tried, in Colab, with LoRA. The loss never came down far enough, and a system prompt on the base model beat it outright. That question is closed. Do not reopen it without a new reason.

§9.2 `THINKING_CONTEXT` and the tone additions

Three optional blocks get appended to the context when they apply:

Frustration. `_track_frustration` in `core/app.py` watches for signs you are annoyed, and when it fires, this goes in: *"Drop any cheerful energy. Be direct, get to the point faster, skip filler."*

Thinking mode. Triggered by the `think` voice shortcut. Overrides the persona into Socratic mode — questions only, no advice.

The two behavioural rules worth preserving, both already in the persona:

- *Handling gaps.* Two options only: make the most reasonable assumption, act, and say which assumption you made; or, if the choice genuinely changes the outcome, ask **one** short question. Never say you are confused,

never list every interpretation, never freeze. A wrong-but-stated assumption beats a stalled conversation, because you will just correct it.

- *Knowing your limits.* Give a best take and be honest about confidence rather than bluffing.

§9.3 The mode line

```
"CURRENT MODE: CHAT – text in a window, mic off. Trust this line over anything
said about modes earlier. ..."
```

Regenerated fresh every turn, and it is the **only** truth about the current mode.

The explicit "trust this line over anything said earlier" exists because you flip modes mid-conversation. Turn six says "I am speaking aloud"; turn nine is typed. Without the override, the model reads its own earlier claim and formats for the wrong medium.

The voice version bans markdown, lists and code blocks, and asks for numbers as words. The chat version requires fenced code blocks with language tags. Those are exact opposites, which is why they cannot both live in the static persona.

§9.4 Message order, and prompt caching

Here is the assembled list:

```
messages = ([_system] + recent
             + [{"role": "system", "content": context},
               {"role": "user", "content": _user_content}])
```

So the order is:

```
[ static system prompt ][ recent history ][ per-turn context ][ your message ]
```

Not the more obvious `[system][context][history][user]`. The reason is worth understanding properly, because it is the most useful performance fact in the codebase.

Providers cache the beginning of a prompt they have seen before and skip reprocessing it — but only while that beginning is **byte-for-byte identical**.

The context block changes every single turn: new date, new facts, new mode line. Put it early and you change the prefix on every turn, so the cache misses on every turn, so every turn reprocesses the whole prompt from scratch.

Put it late, and the expensive unchanging part — the persona, the tool rules — stays cached and cheap.

There is a second, unrelated benefit: **instructions closest to the user message are followed more reliably**. Recency wins in attention. So the placement that is fastest is also the placement that works best. That is rare and worth taking.

The same reasoning shapes `TOOL_RULES` and `TOOL_GUIDANCE`. They are *concatenated onto the persona* rather than sent as their own message, because for a given shape they are byte-identical every turn and stay inside the cached prefix. And there are exactly **two** shapes — with real tools, and without — not one per turn, precisely so caching still works. A menu holding nothing but `find_tools` cannot chain calls, cannot send a message and cannot lie about having closed an app, so about 360 tokens of rules governing all that are dropped. On "how are you", that was most of the bill.

§9.5 `stable_window` — chunked trimming

Anchor: [BOOK §9.5]

The obvious way to keep the last N messages of history:

```
recent = conversation[-20:]
```

That is wrong here, and for exactly the reason in §9.4. A sliding window shifts by one every turn. The prefix changes every turn. The cache misses every turn.

This was the "fast for four replies, then slow" cliff.

`stable_window` returns a window whose **start** only moves once every eight appends. Between moves the prefix is byte-identical and stays cached; when it moves you pay once, then get another eight cheap turns.

```
recent = stable_window(conversation[1:], history_limit)
```

`conversation[1:]` skips index 0, which is always the system prompt and is handled separately.

`history_limit` varies by scope: 20 for full, 10 for relevant, 8 for operational. That 8 was raised from 4 for a specific reason — an operational turn gets no episodic retrieval, which makes history the *only* place "we were doing Disney songs" can live, and at four messages "play a different one" had already lost the thread and replayed the first song.

Prefix stability is a performance feature. That is the sentence to remember. §34.

Chapter 10 — Making the call

File: `core/providers.py`, 975 lines. **Anchor:** [\[BOOK §10.1\]](#), [\[BOOK §10.3\]](#), [\[BOOK §10.6\]](#)

§10.1 The one door

Anchor: [\[BOOK §10.1\]](#)

Every thought Ted has leaves the program through `chat_create()`. Replies. Tool decisions. Fact extraction. Session summaries. Describing a screenshot. All of it. There is no second path.

That is not tidiness for its own sake. Ted used to name models in four separate places, and they drifted apart — different files believing different things about which model was current. Now a model name enters a request in exactly one file, and everything else asks for thinking and gets it.

What `chat_create` does:

1. **Check the pin.** Has the header dropdown forced cloud or local? §10.6.
2. **Unless pinned local, try Groq.**
3. **If Groq fails for any reason at all** — no key, rate limit, 500 error, dropped connection, timeout — retry the *identical* request against Ollama on this Mac. §10.4.
4. **Record which one answered**, so the window's health dot can be honest.

Callers never classify errors. Nothing else in Ted asks "was that a rate limit or an outage?" That is the entire value of this file, and it is why adding a second path anywhere else would be a real regression rather than a style question.

§10.2 `max_retries=0`, and why

Right below the imports:

```
_groq = Groq(api_key=GRQ_API_KEY, max_retries=0) if GRQ_API_KEY else None
```

The Groq SDK retries twice by default, silently, with backoff — and its timeout is **per attempt**. So a request you configured for thirty seconds can take well over a minute, with no output, no error, and no way to tell it apart from a hang.

That is not hypothetical. From a real log: `request accepted after 40939ms (groq)`. Forty-one seconds inside a single `create()` call.

Retrying is fine. It belongs somewhere that can say what it is doing. Below this line, a rate limit falls through to the local brain, and failing that surfaces as an error you can read within a second or two.

Do not put the retries back.

§10.3 The Ollama path

Anchor: [BOOK §10.3]

Ollama is a separate program on your Mac that runs models locally. Ted talks to it over HTTP on `127.0.0.1:11434`.

The response shape is different from Groq's, so `_OllamaStream` and `_ollama_create` translate it — using `SimpleNamespace` to fake the object shape the rest of Ted already knows how to read. That is why nothing above this file has to care which brain answered.

`_ensure_ollama` has a budget, and the budget is the point. An earlier version polled twenty times at a one-second timeout. Ollama is installed on your Mac but never starts on its own, so that cost about **twenty-three seconds per message**, every message, for nothing. There is now a six-second budget and a five-minute cooldown after a failure. A dead local brain costs you once rather than continuously.

§10.4 The handover

Groq fails mid-use; the same request is retried locally. Simple in principle, and it has one sharp edge:

A cold Ollama model loads from disk, and that can take up to three minutes. `_ollama_create` allows a 180-second timeout for exactly this. Three minutes of silence reads as a crash.

Hence:

```
llm.providers.set_fallback_notice(self._announce_local_handover)
```

`core/app.py` registers a callback at startup so the window can say "switching to the local brain" rather than going quiet. If you ever make this path quieter, you are recreating a bug that looks exactly like a freeze.

Status, honestly: the fallback logic is tested and the local model (`qwen3.5:35b-a3b`, about 24 GB) has been pulled and verified on your Mac. The *moment of handover* — Groq failing in real use and the local model taking over — has never been watched and timed. §35.

§10.5 Rate limits and the cooldown

Groq reports your account's ceiling and what is left of it in the headers of every response. `_read_rate_headers` and `_note_usage` keep track.

When a rate limit hits, `_start_cloud_cooldown` marks the cloud as off-limits for a while — starting at 15 seconds and backing off to a maximum of 120. That stops Ted hammering a limit it already knows it has hit.

`rate_limit_status()` and `cloud_cooldown_remaining()` feed the diagnostics panel, so "why is it using the local brain right now" has an answer you can look at.

§10.6 The pin

Anchor: [BOOK §10.6]

The dropdown in the window header writes `"auto"`, `"cloud"` or `"local"` into `data/runtime.json`, so it survives a restart.

A pin is obeyed, not preferred. "Cloud" means cloud, and if the cloud is down you get an error rather than a silent local answer. That is deliberate: a forced mode that quietly does something else is not a forced mode, and the whole reason to pin is usually that you are testing one specific path.

`active_provider()` reports which one actually answered the last call — `groq`, `ollama`, or `none` — and the header shows it. So "auto" is honest about falling back.

One subtlety, fixed in August and worth knowing: `groq_ok()` means "the last call was not served locally", **not** "Groq was reached". A fresh session with no completions yet reports healthy. That is deliberate — the previous version inferred "cloud down" from "no calls yet" and cried wolf at every boot.

Chapter 11 — Streaming, and tools

§11.1 `_stream_turn`

The model's response arrives in pieces. `_stream_turn` reads them and sorts them into two kinds:

- **text** — yielded straight through to the caller, which speaks and displays it immediately
- **tool calls** — accumulated, because a tool call arrives in fragments too (the name first, then the arguments a few characters at a time as JSON)

Both can arrive in the *same* response. The model can say "Let me check that" and call a tool in one breath.

That combination caused a real bug: text streamed alongside a tool call was missing from the stored turn, so Ted's memory of what it said differed from what it actually said. Two places owning one fact. §34.

§11.2 `ToolRuntime`

Anchor: [BOOK §11.2]

A small holder for three things that must travel together:

```
_runtime = llm.ToolRuntime(  
    schemas=_selected_schemas,      # what this turn may use  
    dispatch=_selected_dispatch,      # how to actually run one  
    action_tools=th.ACTION_TOOLS,     # which count as real changes  
    on_failure=_note_action_result,  
    is_failure=th.looks_like_failure,  
)
```

`action_tools` is there so the honesty check knows whether anything actually happened (§11.8). `add_schemas()` is how `find_tools` grows the menu mid-turn (§7.3).

§11.3 The tool loop

```
model responds  
├─ text only?      -> stream it, done. One round trip.  
└─ tool call?      -> run it  
                     feed the result back as a message  
                     ask again  
                     (repeat, up to MAX_TOOL_ROUNDS = 5)
```

Bounded at five rounds and ten total calls. Without a bound, a model that keeps reaching for tools loops until something else stops it.

This used to be two calls. The old design made a cheap non-streamed "probe" first — asking "does this need a tool?" — threw that answer away, and then made the real streaming call. Two round trips on every single message, when the overwhelming majority of messages are conversation.

[timing] `tool probe` should never appear in the log again. If it does, you are on the legacy path (`TED_LEGACY_LADDER=1`).

`tool_choice` stays **"auto"** on the first call, even when the router is confident this is an action. Forcing it removes the model's only honest escape route: a turn that cannot be satisfied by any loaded tool would have nowhere to go but a wrong call. The recovery path forces tool choice on a *retry* — after the model has demonstrably narrated an action instead of taking one, which is evidence rather than a guess.

§11.4 `core/tools.py` — the menu

Anchor: [BOOK §11.4]

One list, ~1,330 lines, no logic. Each entry describes one tool.

```
{
  "type": "function",
  "function": {
    "name": "open_app",
    "description": "Open an application on the Mac. For a website use browse_to.",
    "parameters": {
      "type": "object",
      "properties": {
        "name": {"type": "string", "description": "The app to open"}
      },
      "required": ["name"]
    }
  }
}
```

The description is the only thing the model knows about a tool. It does not read your Python. It reads that sentence and decides from it. A vague description produces a tool used at the wrong moments, and no amount of code elsewhere fixes that.

Two habits that make descriptions work:

- **Say when to use it, not what it does.** "Use when the user asks to open an application" beats "Opens an application."

- **Say what to use instead.** Half of a good description is fencing it off from its neighbours. That is why `open_app` mentions `browse_to`.

The current menu, as of this writing:

```
open_app      close_app      browse_to     play_youtube
web_search    show_image    now_playing   search_chats
play_music    play_playlist spotify_control add_to_playlist
remove_from_playlist create_playlist delete_playlist
send_message  set_reminder  set_timer     get_reminders
get_weather   get_emails    read_email    email_action
send_email    search_knowledge add_knowledge  calendar_get
calendar_add  notes_add     notes_get     clipboard_read
clipboard_write system_volume system_brightness screen_describe
ui_inspect    ui_press      ui_fill       create_document
type_text     press_key     scroll         calculate
log_habit     get_habit_streak learn_lingo    clarify_lingo
bouncer_watch bouncer_status bouncer_toggle text_respond
code_overview code_search    code_read     code_tree
code_history   code_diff     code_write
notebook_read  notebook_write notebook_edit  notebook_delete
notebook_search
```

Plus `find_tools`, which is always present and is not in this list because it is defined in `core/routing.py`.

§11.5 `core/tool_handlers.py` — the doing

The kitchen. Where several of those tools actually happen, and where two rules are enforced.

Note the pattern: **every handler returns a plain string**, and that string is what Ted says. There is no separate status-code layer. That is unusual and it is deliberate — it makes it structurally hard to throw away what really happened.

§11.6 `_dispatch_tool` — the switchboard

Anchor: [BOOK §11.6] in `core/app.py`.

The model has chosen a tool by name and supplied arguments as a dictionary. This method turns that into something happening.

It is one very long chain of `if name == "...":` branches. That is not elegant and does not need to be — it is a lookup table written in if-statements, and its only jobs are to be complete and obvious.

This is the step people forget when adding a tool. A schema in `core/tools.py` with no branch here means the model calls something that silently does nothing, and the failure is invisible because the model cheerfully narrates

success. Chapter 31.

Two rules every branch follows:

```
args.get("name", "")    # never args["name"] – optional args may be absent
return "Opened Spotify." # the sentence Ted says, and it must be true
```

§11.7 Confirmation

Anchor: [BOOK §11.7]

`needs_confirmation(name, args)` decides whether Ted asks first.

Note it takes the **arguments**, not just the name. Crossing out one notebook entry is an ordinary edit; deleting a whole page throws work away. A plain "is this tool on the dangerous list" check cannot tell those apart, which is why the older `name in CONFIRMATION_TOOLS` version was replaced.

Both places in `core/app.py` that gate an action call this one function. Two callers disagreeing about whether something was consequential is the duplicated-judgment bug again. §34.

When a tool needs confirmation, `_pending_tool_confirmation` is armed and `_respond` returns. Your next message is caught by rung 5 (§6.3) and read as the answer. The confirmation quotes the message back to you — you were once asked to approve a *blank* one and reasonably objected.

Currently unconfirmed: `type_text`, `clipboard_write`, and app control. Low risk today because Ted has no browser automation. It stops being low risk the moment that lands. §35.

§11.8 The honesty rule

Anchor: [BOOK §11.8]

This is the rule the persona never breaks, and it exists because of two real incidents of the same shape: **intent reported as outcome**.

Incident one. `play_track` said "Playing X" whenever Spotify's API did not raise an error. The Web API accepts that call and returns success in plenty of cases where nothing plays. Now `_confirm_playing` polls what is actually playing and returns True, False, **or None** — and None is deliberately not treated as success.

Incident two, worse. Ted said "*Closed VS Code and Notes.*" having called no tool at all, then insisted it had no way to close apps — while `close_app` was in its menu and already verified to quit apps properly. Every safeguard in the codebase was downstream of the failure. Nothing was watching for the model simply narrating an action it never took.

Three pieces now enforce this:

ACTION_TOOLS — a frozen set of names that make a real change in the world. Everything else only looks things up.

`claims_completed_action(text)` in `core/llm.py` — looks for a past-tense action claim ("closed", "sent", "played") in a turn where no ACTION tool ran. When it finds one, a correction is appended and the incident is logged with a `[honesty]` tag you can grep for.

`looks_like_failure(result)` — one shared answer to "did that go wrong?", used by every caller: the window's issue toast, the routine runner, the reflex runner, the telemetry row. One function, because two pieces of code answering that question differently is this codebase's recurring bug.

Do not weaken any of this in a refactor. It is the one rule stated as non-negotiable, and it is what stops Ted being confidently, cheerfully untrue about your own computer.

Chapter 12 — Speaking and showing the answer

§12.1 `speak_streaming`

File: `core/voice.py`.

```
full, barged = speak_streaming(w, gen, self, speed=resp_speed, volume=resp_vol)
```

It takes the generator from `ask_streaming` and does three things with each chunk as it arrives:

1. pushes the text into the window
2. accumulates it into `full`
3. when a complete sentence has formed and voice is on, synthesises and plays it

`_find_sentence_break` decides where a sentence ends. Speaking sentence by sentence rather than waiting for the whole reply is why Ted starts talking almost immediately.

It returns two things. `full` is the complete reply, which becomes `self.last_reply` and gets stored. `barged` is True if you talked over Ted, which propagates back up through `_respond` so the conversation loop knows the turn was interrupted.

Two small touches worth noticing: reply speed is adjusted by content type (`_classify_content_speed`), and volume drops to 50% when you are whispering.

§12.2 The bridge, Python side

File: `core/hud_bridge.py`, 58 lines. **Chapter 20** covers the other half.

Four functions, and they are worth reading in full because they are the entire Python-to-window vocabulary:

```

def js(window, code):
    try:
        window.evaluate_js(code)
    except Exception:
        pass

def set_state(window, s):
    js(window, f"tedHud.setState('{s}')"")

def add_message(window, role, text):
    js(window, f"tedHud.addMessage('{role}', {json.dumps(text)})")

def show_issue(window, text):
    js(window, f"tedHud.showIssue({json.dumps(text)})")

```

Python does not manipulate HTML. It **runs a line of JavaScript** in the page, and that line calls a function on the `tedHud` object.

Two details:

`json.dumps(text)` rather than pasting the text in. That handles quotes, newlines and backslashes correctly. Without it, a reply containing an apostrophe would produce broken JavaScript and silently do nothing.

except Exception: pass. These are called from audio threads. A JavaScript error, or a window that has not finished loading, must never be able to kill the thread listening to your microphone. This is one of the few places where swallowing an error is right.

The states `set_state` can push: `idle`, `listening`, `thinking`, `speaking`, `error`. They drive the particle sphere's colour.

Chapter 13 — After the turn

Three things happen once the reply is finished, and all of them are meant to stay off the critical path.

§13.1 Saving the exchange

`_remember_exchange` writes the message and the reply into the `exchanges` table, which is FTS5-indexed and therefore searchable later. On a background thread.

The user turn is appended to the in-memory `conversation` list **only after a successful reply**, so a failed call does not leave a dangling user message with no assistant response beside it.

§13.2 Fact extraction

Anchor: [BOOK §13.2]

Ask a model to pull `Charlie / LIVES_IN / Spirit Lake` out of a sentence, and store it.

Two pieces of history made this function what it is.

It was dead for five weeks and nobody knew. It asked for JSON, got prose back, `json.loads` raised, and the exception died inside a `print()`. The facts table had one row. Now: JSON mode is requested explicitly, there is a salvage parser for near-misses, and real failures go to `ted_errors.log`. This is the origin of "silent failures are the expensive ones". §34.

It used to harvest world knowledge out of Ted's own replies. It once saved "bananas are berries" as a fact about you. Two defences now: a hard rule in the prompt, and a Python gate that rejects any fact whose subject never appeared in what **you** said and is not you.

Timing changed in August, and the reason is good. Extraction used to start after Ted stopped talking, and took about seven seconds on the local brain — so "Memory updated" landed ten seconds after you said the thing being remembered, which reads as Ted not having noticed. Nothing in the extractor needs the reply. Started at the *top* of `ask_streaming`, it runs alongside the reply and the toast usually beats Ted's last sentence.

§13.3 Session summaries

Anchor: [BOOK §13.3]

At the end of a conversation Ted may write a short dated first-person memory of it. Those get injected into later replies, so callbacks land mid-conversation — *"yesterday you were stuck on that double-firing webhook"* — rather than only in a greeting.

Most conversations produce no memory at all, and that is correct.

Two filters. A cheap Python pre-filter (`session_has_substance`) counts words in *non-routine* turns only. Then the model itself, told explicitly that declining is the right answer most of the time.

Seeing this in the log is the system working:

```
[memory] shutdown: nothing worth remembering this session
```

A memory list full of "Charlie set a two minute timer" makes callbacks **worse** than having none. Selective memory beats complete memory. §34. Do not "fix" this.

Write triggers: ten minutes idle, every twelve exchanges (crash insurance), and on exit. All three **upsert the same row**, so one conversation leaves one memory rather than three fragments. A `kill -9` loses turns since the last flush, and that is accepted.

Tuning knobs if it genuinely misbehaves: `MIN_MEMORY_SUBSTANTIVE_WORDS` (currently 15) and `_ROUTINE_OPENERS` , both in `core/llm.py` .

§13.4 Telemetry

File: `core/telemetry.py` .

One database row per turn: which brain answered, how many tokens, where the wait went, what failed, which tools were offered and which were used.

This exists because the answer to "why was that slow" used to live only in a terminal, only while it was open, and only if you happened to be looking. Now it is queryable after the fact — which is the difference between debugging and guessing.

Three rules it follows, because it sits on the reply path:

1. **It never raises into Ted.** Every public function swallows its own errors. A telemetry bug must not be able to break a conversation.
2. **It never blocks.**
3. **It records what happened, not what was intended.** Same rule as the tools.

The Diagnostics panel reads this. `_turn.ctx_breakdown` is particularly worth knowing about — it records where the prompt tokens actually went, per block:

```
persona=470;facts=180;recall=95;history=340;tools=910;other=210
```

That is the difference between "'how are you' cost 1,782 tokens" and knowing which block to cut.

PART III — MEMORY

Chapter 14 — `core/memory.py`

File: 874 lines. **Anchor:** [\[BOOK §14.2\]](#) , [\[BOOK §14.5\]](#)

§14.1 One file on disk

Everything Ted knows about you that survives closing the window lives in `data/memory.db`. It is a **SQLite** database, which means it is not a server — it is a single file you talk to in SQL. Nothing to install, nothing to start, and nothing to forget to start.

That last part is why it exists. The previous backend was **Neo4j**, a graph database that needed Neo4j Desktop running. It usually was not, and Ted silently fell back to in-session memory only. Replaced on July 1.

The tables:

Table	What it holds	Read by
<code>facts</code>	statements about you, subject/relationship/object	every prompt
<code>exchanges</code>	a log of turns, FTS5-searchable	retrieval, per turn
<code>session_summaries</code>	first-person memories of whole conversations	retrieval
<code>chat_sessions</code> / <code>chat_turns</code>	the dashboard's chat threads	the sidebar
<code>notebook_pages</code> / <code>notebook_entries</code>	Ted's notebook	Chapter 16
<code>lingo</code>	your shorthand	routing, every turn
<code>routines</code>	phrase-to-action definitions	rung 7
<code>bouncer_rules</code> / <code>bouncer_state</code>	who gets announced	the bouncer
<code>turn_log</code>	telemetry, one row per turn	the Diagnostics panel
<code>memory_audit</code>	every memory write, via SQLite triggers	the History tab
<code>audit_context</code>	one row: who is writing right now	the triggers
<code>patterns</code>	topic counts by hour	nothing
<code>habit_logs</code>	habit streaks	nothing
<code>goals</code>	leftover from a deleted feature	nothing

The last three are honest dead weight. §35.

§14.2 Facts, and the rule about them

Anchor: [BOOK §14.2]

A fact is a triple:

```
Charlie | LIVES_IN | Spirit Lake, Iowa
```

Anything you say with *I / my / we* is a candidate. Impersonal content goes to the knowledge base instead (Chapter 17) and is searched on demand rather than injected always.

Facts are in every prompt. That is the whole point of them, and it is also why this table being full of junk costs you on every single message.

Facts supersede, they do not stack. For a single-valued relationship — where you live, how old you are — a new value *replaces* the old one.

Without that, both `LIVES_IN Spirit Lake` and `LIVES_IN Spirit Lake, Iowa` sat in the table, and therefore in every prompt, forever. Ted read both every turn and sounded confused about something you had told it clearly.

When two versions differ only in specificity, the more specific wins. `_norm_rel` and `_norm_obj` are how "the same thing said differently" is recognised.

`forget_fact` and `list_facts` exist, and the Memory panel is the easy way to use them. Editing a wrong fact directly is almost always better than trying to talk Ted out of it.

§14.3 `exchanges` and FTS5

FTS5 is SQLite's built-in full-text search: a virtual table kept in sync with the real one, letting you search *words* rather than match strings.

That is what makes `get_memory(query)` work — you say something about a webhook, and Ted can find the turn three weeks ago where you were debugging one, even though nothing about the wording matches.

There is a fallback chain, and it matters: FTS5 if available, `LIKE` keyword matching otherwise, and **the most recent exchanges if nothing matches at all**. So there is always *some* grounding context rather than an empty block.

§14.4 Session summaries

Covered in §13.3 from the writing side. From the reading side: `get_recent_memories` pulls them with an age limit, and `format_memories_for_prompt` turns them into the block that goes into the context.

`_humanize_date` turns a timestamp into "yesterday" or "last Tuesday", because "2026-08-11T19:04:51" in a prompt produces a reply that sounds like a database.

§14.5 `memory_event` — one emitter

Anchor: `[BOOK §14.5]`

Every memory write comes through here so the window can show one toast:

```
Memory updated: Charlie likes Chick-fil-A
Memory removed: Charlie lives in Spirit Lake
```

Explicit writes, background extraction, session summaries — all three paths, one emitter.

One emitter is the point. Two places deciding what counts as a memory event is the duplicated-judgment bug that keeps biting this codebase. §34.

The toast is clickable and opens the Memory panel scrolled to that row, so fixing something Ted got wrong is two clicks rather than a conversation.

§14.6 Why nothing here raises

Read the module docstring: every function degrades gracefully and returns `[]`, `""`, `None` or `False` rather than raising.

Memory is context. It is not the answer. A broken database should make Ted less informed, never make Ted stop working. Keep that property if you add functions here — it is a load-bearing promise that the rest of the codebase relies on without checking.

§14.7 If you change the schema

Two places, always:

1. the `CREATE TABLE IF NOT EXISTS` block, for a fresh database
2. `_migrate(conn)`, for a database that already exists

Skipping `_migrate` works perfectly on your machine — where the table gets created fresh — and breaks on any machine that already has data. Which is, in practice, yours, the day after.

§14.8 Practical SQL notes

```
conn.execute("SELECT * FROM facts WHERE subject=?", (subject,))
```

The `?` is a placeholder; the tuple supplies the value. **Never build SQL by pasting text together** with `+` or an f-string. This is not a style preference: a value containing a quote character would change the meaning of the statement.

WAL mode (write-ahead logging) lets one writer and several readers work at once. Ted has many threads; without it they collide.

`threading.local()` gives each thread its own connection, because SQLite connections cannot be shared across threads.

To poke at the database yourself:

```
sqlite3 -box ~/ted-ai/data/memory.db "SELECT * FROM facts"
sqlite3 -box ~/ted-ai/data/memory.db ".tables"
```

Chapter 15 — The dashboard

Files: `dashboard/app.py` (691 lines), `dashboard/db.py` (616), plus three HTML pages. **Anchor:** [BOOK §15]

§15.1 What it is

A small **Flask** web server on `127.0.0.1:5175`. Flask is a Python library for writing web servers: you write a function, put a decorator above it naming a URL, and that function now answers requests to that URL.

```
@app.route("/api/chats")
def api_chats():
    return jsonify(...)
```

The Memory, Notebook and Diagnostics panels inside Ted's window are **web pages in an iframe**, served from here. You can also open them in a normal browser:

```
open http://127.0.0.1:5175          # Memory
open http://127.0.0.1:5175/notebook
open http://127.0.0.1:5175/diagnostics
```

Or run it standalone with Ted closed:

```
cd ~/ted-ai && python -m dashboard
```

§15.2 What it can do

Full create/read/update/delete over the memory tables, chat session storage and search, the provider pin, routine and lingo editing, and the diagnostics views.

The route names in `dashboard/app.py` read as a table of contents: `api_rows`, `api_create`, `api_update`, `api_delete`, `api_chat_search`, `api_diag_turns`, `api_provider_set`, `api_routine_create`, `api_lingo_list`, and so on.

`/api/version` is worth knowing about — it exposes capability flags, and `hud.py` checks them at startup. If an *old* dashboard is holding port 5175, chat history would silently fail to save, so Ted prints a loud warning. §4.3.

§15.3 The table registry

File: `dashboard/db.py`.

For each table it whitelists which columns are readable, editable and searchable. A web form cannot reach a column that is not on that list.

That is the safety layer. Without it, "let the dashboard edit memory" quietly becomes "let anything that can reach port 5175 edit anything".

§15.4 The audit log, and the clever bit

Two design decisions here are genuinely good, and both are the kind of thing a tidy-minded refactor would destroy.

The audit log is a set of SQLite triggers, not dashboard code. The triggers are stored *inside the database file*.

Which means they fire for **Ted's own writes too** — from a completely different process — not just for edits made through the web page.

Putting the audit in the dashboard would have logged half the writes and looked complete. That is worse than no audit, because you would trust it.

Actor attribution. A one-row `audit_context` table records who is writing, defaulting to `'ted'`. The dashboard flips it to `'user'` **inside its own uncommitted transaction**, writes, then flips it back before committing.

Because the change is never committed while set, a concurrent write from Ted's process cannot see the `'user'` flag and be mislabelled. Two processes, one database, correct attribution, no locking.

If a refactor makes this simpler, it has almost certainly made it wrong.

Chapter 16 — The notebook

File: `core/notebook.py` , 423 lines. **Anchor:** `[BOOK §16.3]`

§16.1 What it is, and what it is not

Named pages of numbered entries that Ted owns and can read, write, edit and delete.

It is deliberately **not** two things it is easy to confuse it with:

- **Apple Notes** (`core/notes.py`) is *your* app. Ted is a guest there.
- **The knowledge base** (Chapter 17) is searchable but not revisable.

The notebook is Ted's, it is structured, and every operation on it is exact — no embeddings, no similarity, no guessing which page was meant.

§16.2 A page is a list, not a blob

This is the load-bearing decision.

A page is an **ordered list of entries**. An entry is one thing that got written down, stamped with when and by whom.

With a blob of text, changing one line means Ted rewriting the whole page from memory — and a model rewriting a page it only half remembers is how notes quietly lose content. You would not notice for weeks.

With numbered entries, an edit names exactly one row and **physically cannot touch the rest**.

If you are ever tempted to store pages as one string, this is the paragraph to re-read.

§16.3 The index line

Anchor: `[BOOK §16.3]`

Two mechanisms, deliberately split:

`index_line()` is in every prompt. Page names and entry counts only. Never contents. So Ted can neither invent a page nor deny one that exists. It is one local database read of a table with a handful of rows, and it returns `""` when the notebook is empty — so an unused feature costs zero tokens.

`notebook_read` is a tool call. Contents cost a read, always. The persona says it plainly: what is on a page, he reads — never from memory, never paraphrased from an earlier turn.

Index = the map. Read = the territory. Neither alone would have been enough: the index alone lets Ted guess at contents, and the read alone lets Ted deny a page exists because it did not think to look.

§16.4 Page names are cleaned

"my fixes page", "the fixes notes", "FIXES" and "fixes" are one page. `_clean_name` does that.

The model does not have to normalise, and — more importantly — two phrasings cannot silently become two pages. If you find duplicates appearing, `_clean_name` is not stripping the wording you actually use.

The panel is `dashboard/notebook.html` : ruled lines at 30px, text line-height 30px so writing sits *on* the lines, and a handwriting face from whatever the Mac already has (Bradley Hand, falling back to Chalkboard SE). No web font, deliberately — Ted has to look right offline.

Chapter 17 — The knowledge base

File: `core/knowledge.py` , 325 lines.

§17.1 A different kind of memory

`core/memory.py` stores facts in SQL tables and finds them by matching words. This stores documents and finds them by **meaning**.

Text is converted into a list of numbers — an **embedding** — that captures roughly what it says. Searching converts your question the same way and finds the stored pieces whose numbers are closest.

That is why it can match "how do I stop it freezing" against a paragraph that never uses the word "freezing".

The store is **ChromaDB**; the embeddings come from **fastembed**, which uses ONNX rather than PyTorch — a much smaller install for the same job.

§17.2 Chunks, and why they overlap

Documents are split into overlapping pieces.

The overlap is not an implementation detail. A sentence that answers your question might straddle a boundary; without overlap it would be half in each chunk and findable in neither.

§17.3 Getting things in

```
cp ~/Downloads/syllabus.pdf ~/ted-ai/inbox/
```

then say *"index my documents"*.

Long attachments also land here automatically (§27.1), so a PDF you dropped into the chat stays askable after that conversation has scrolled away.

It is loaded at startup on a thread, because it used to be the slowest of the five retrieval loaders and regularly ate the whole four-second budget on the first message of a session. §8.2.

Chapter 18 — Lingo

File: `core/lingo.py` , 206 lines.

§18.1 Why it is separate from facts

A **fact** adds context to an answer.

A **lingo mapping changes how Ted interprets your words** — everywhere: routing, routines, and the compact operational context the model sees.

Different job, so different table and a different point in the ladder.

"When I say *the dispatch app* I mean the crew scheduling project" is not a fact about you. It is a translation rule, and it has to be applied *before* Ted decides which tools a message needs — otherwise the router is reading words it does not understand.

§18.2 Where it happens

Rung 6 of the ladder (§6.4) catches a definition and stores it immediately, without waiting for background fact extraction to eventually catch up.

Just below, every message runs through `lingo.expand()` :

```
routing_text, matched_lingo = lingo.expand(text, record_usage=True)
```

`routing_text` — your message with shorthand replaced — is what all the routing rules below see. Your **original** text still goes to the model, along with a short context line saying what the shorthand meant.

That split is right: the router needs the expansion to do its job, and the model should hear you in your own words.

`clarify_lingo` is the other direction — a tool the model can call when it meets a term it does not know, which arms `_pending_lingo` and asks you.

PART IV — THE WINDOW

Chapter 19 — `ui/ted_hud.html`, the shape of it

File: 2,068 lines. **Anchor:** the block at the top of the file.

§19.1 One file, no build step

The entire window — layout, every style, and every line of JavaScript — is in this one file. There is no framework, no npm, no build step. You edit it, you restart Ted, you see the change.

That trade buys simplicity at the cost of length. The file is meant to be **navigated by searching, not by scrolling**.

§19.2 The three sections

Lines	Tag	What
~80 – 558	<code><style></code>	All the CSS
~560 – 706	<code><body></code>	The markup
~707 – 2066	<code><script></code>	The behaviour, and half the bridge

Colours are defined once, at the very top, as CSS variables:

```
:root{
  --bg:#151717; --side:#1a1d1d; --panel:#212525;
  --text:#e9edec; --dim:#939b99;
  --accent:#16DEDE;           /* teal */
  --ok:#5fc492; --warn:#dcab50; --down:#d95850;
}
```

Used everywhere as `var(--accent)`. Change one line there and it changes throughout. Do not hardcode a hex value anywhere else in the file.

§19.3 The layout, by element id

Searching for `id="..."` is how you find anything in here.

Left sidebar — `#side` `#newchat`, `#chatq` (search), `#chats` (the list), `#sidefoot` holding `#membtn`, `#diagbtn`, `#notebtn`.

Top bar — `#top` `#np-stack` (now playing) with `#np-skip`, `#chat-title`, `#clock` (`#clk-time`, `#clk-date`, `#clk-wx`), `#brainpick` (the provider pin dropdown), `#budget` (`#brain` dot, `#bud-bar`, `#bud-txt`), `#status`.

The middle — `#scroll` > `#msgs`, with `#empty` shown when there are no messages.

The input row — `#inwrap` > `#bar` `#mic`, `#attach`, `#input` (the textarea), `#transcribe`, `#send`. Plus `#attachments` above it and `#heard` for the live transcription flash.

Right panel — `#apps` `#apps-summary`, `#apps-list`. Collapsed by default.

Overlays — Memory, Notebook and Diagnostics, each a full-screen panel with an `iframe` pointing at the dashboard. They are hidden until opened, and opening one closes the other two.

`#drop` — the full-window drop target for attachments.

§19.4 Debugging it

You cannot open developer tools the usual way.

`console.log` still works, and Python can read it back. When something in here silently does nothing, the cause is almost always one of two things:

1. a typo in an element id passed to `$()`, so it returned `null`
2. a Python call to a `tedHud` function that no longer exists

Both fail silently, which is why they are worth checking first.

Chapter 20 — The `tedHud` object, and the bridge

§20.1 The two channels

Python and this page are separate worlds. They share no variables. They talk in exactly two ways, and nothing else crosses.

JavaScript to Python:

```
window.pywebview.api.ask("what's the weather")
    .then(function(reply){ ... });
```

Calls a method on the `TedApi` object. Returns a **Promise** — an object representing an answer that has not arrived yet — so you use `.then()` to do something with the result when it does. Only methods that exist on `TedApi` can be called.

Python to JavaScript:

```
js(window, "tedHud.setState('thinking')")
```

Python runs a **line of JavaScript** in the page, and that line calls a function on `tedHud`.

§20.2 `tedHud` is a contract

Search for `var tedHud={` — around line 1352.

Every function on that object is part of a contract with Python. The current set:

<code>setState</code>	<code>addMessage</code>	<code>streamTedText</code>	<code>endTedReply</code>
<code>showIssue</code>	<code>clearIssue</code>	<code>memoryEvent</code>	<code>showMedia</code>
<code>incomingText</code>	<code>setMuted</code>	<code>setTranscribing</code>	<code>fillInput</code>
<code>flashHeard</code>	<code>addTimer</code>	<code>clearTimer</code>	<code>clearTimerById</code>
<code>flashAlarm</code>	<code>showReminders</code>	<code>hideReminders</code>	<code>setHealth</code>
<code>setNowPlaying</code>	<code>setOpenApps</code>	<code>setComputerState</code>	<code>noteAppUse</code>
<code>showMemory</code>	<code>hideMemory</code>	<code>toggleMemory</code>	
<code>showNotebook</code>	<code>hideNotebook</code>	<code>toggleNotebook</code>	
<code>showDiagnostics</code>	<code>hideDiagnostics</code>	<code>toggleDiagnostics</code>	

plus a run of no-op stubs at the bottom.

Renaming one silently breaks the Python side. Python sends a *string* of JavaScript; nothing checks it until it runs, and `js()` swallows the error. So if you rename, grep the Python:

```
grep -rn "setNowPlaying" ~/ted-ai/core/
```

§20.3 The no-op stubs

Near the bottom of `tedHud` :

```
pushAmplitude:function() {}, clearAmplitude:function() {},  
setAttention:function() {}, micIdle:function() {},  
setMode:function() {}, setVoiceMode:function() {}, toggleMode:function() {},
```

Functions that do nothing. They are leftovers from the voice-orb era that Python may still call, and they exist so a stale call cannot throw an error inside the window.

Leave them. They cost nothing and they are cheaper than auditing every Python call site.

§20.4 Streaming into the window

`addMessage` appends a finished message. `streamTedText` appends a *piece* of one and `endTedReply` closes it off. That pair is what makes Ted's reply appear word by word rather than all at once.

`memoryEvent` is the toast for a memory write (§14.5), and it is clickable — it opens the Memory panel scrolled to that row.

Chapter 21 — Adding a button, end to end

A worked example. Say you want a button in the sidebar that clears the current chat.

Step 1 — the markup

Find `<div id="sidefoot">` (around line 574) and add a line:

```
<div id="clearbtn" title="Clear this conversation">&#128465; Clear</div>
```

`🗑` is an HTML entity — a character written by its number, so the file stays plain ASCII. Look up the number for whatever symbol you want.

Step 2 — the style

Find the rule listing the other sidebar buttons and add yours:

```
#membtn,#diagbtn,#notebtn,#clearbtn{cursor:pointer;padding:6px 0;
  color:var(--dim);transition:color .2s}
#membtn:hover,#diagbtn:hover,#notebtn:hover,#clearbtn:hover{color:var(--text)}
```

Note `var(--dim)` and `var(--text)`, not hex codes. §19.2.

Step 3 — the click handler

Down in the `<script>` section, near the other `.onclick` assignments:

```
$('#clearbtn').onclick=function(){
  if(!inApp()) return; // running outside pywebview
  window.pywebview.api.clear_chat().then(function(r){
    if(r && r.say) tedHud.showIssue(r.say);
  }).catch(function(){ tedHud.showIssue('Could not clear the chat. '); });
};
```

`$()` is the shorthand helper defined near the top of the script — `document.getElementById`. `inApp()` guards against the page being opened outside Ted, where `window.pywebview` does not exist.

Step 4 — the Python method

In `core/app.py`, near the bottom, in the JS API section:


```
def clear_chat(self):
    """Start a fresh conversation without restarting Ted."""
    self.active_conversation = [self.active_conversation[0]] # keep the system prompt
    js(self.window, "tedHud.clearMessages()")
    return {"ok": True, "say": "Cleared."}
```

Two things to notice.

Keeping index 0. `active_conversation[0]` is always the system prompt. Drop it and Ted loses its personality mid-session, which is a confusing bug to chase.

Returning a dictionary with a `say` key. That is the shape the rest of the JS API uses, so the window can show whatever went wrong rather than a generic failure.

Step 5 — the missing piece

That handler calls `tedHud.clearMessages()`, which does not exist yet. Add it to the `tedHud` object:

```
clearMessages:function(){
    $('msgs').innerHTML='';
    $('empty').style.display='';
},
```

This is the step that gets forgotten, and the symptom is instructive: the chat clears in Python, the window keeps showing the old messages, and nothing anywhere reports an error — because `js()` swallowed it (§12.2).

Step 6 — restart and check

```
cd ~/ted-ai && source venv/bin/activate && python hud.py
```

Watch the terminal. If the button does nothing, check in this order: the id in `$()` matches the id in the markup; the Python method name matches exactly; the `tedHud` function you are calling actually exists.

PART V — VOICE AND AUDIO

Chapter 22 — `core/voice.py`

File: 728 lines. **Anchor:** [BOOK §22.3]

§22.1 Importing this file is not free

It loads speech models and starts the audio engine. It is the runtime, not a library of helpers. The pure, testable text handling lives in `core/intents.py`.

§22.2 The two directions

Hearing. `capture()` records a turn and transcribes it — through Groq's hosted Whisper (`whisper-large-v3-turbo`) by default, or a local Whisper if `USE_GROQ_STT = False` in config. Local Whisper is the offline path.

Speaking. `synth()` turns text into audio with **Kokoro**, a local text-to-speech model running through ONNX, voice `am_michael`. ElevenLabs is available behind `USE_ELEVENLABS` and is not used — the free tier was exhausted, which is what forced local TTS in the first place. That turned out well.

Voice cloning was tried and abandoned. Coqui, XTTS-v2, OpenVoice. Audio quality was too poor. Kokoro is the answer.

§22.3 The gates in `capture()`

Anchor: [BOOK §22.3]

This is the interesting part of the file, and every gate exists because of a specific real failure.

Whisper hallucinates. In a silent room it will confidently return "Thank you", because that is the phrase most likely to follow silence in its training data. Coughs came back as "Tep." and "Start." — and were **executed as commands**.

So audio is refused in stages before any of it is believed:

Gate	Rejects
duration + loudness (RMS)	too short or too quiet to be a turn
Whisper's no-speech score	the model's own reported doubt
<code>_looks_hallucinated()</code>	exact-match blocklist of phantom phrases
<code>_is_junk_fragment()</code>	one-word noise that reads as a command

Deleting a gate brings its failure back. Loosening one is fine, but turn on the debug output and watch real audio first.

`tests/test_capture_gates.py` — 32 checks — pins this behaviour.

§22.4 Speaking while streaming

`speak_streaming()` takes the generator from `ask_streaming` and speaks it sentence by sentence as it arrives rather than waiting for the whole reply. `_find_sentence_break()` decides where a sentence ends.

That is why Ted starts talking almost immediately.

§22.5 Cleaning text for speech

`_clean_for_speech()` strips markdown, expands things that read badly aloud, and `_enforce_contractions()` turns "do not" into "don't" — because written English and spoken English are different registers and the difference is audible.

This is why the mode line (§9.3) bans markdown in voice mode: it is far better for the model not to produce it than for this function to remove it.

If you want to change something:

Symptom	Where
Ted's voice is wrong	the Kokoro voice name, in <code>synth()</code>
Ted talks too fast or slow	<code>SPEED</code> , and <code>adjust_speed()</code>
Ted mishears constantly	the gates in §22.3 — debug first
Ted reads markdown aloud	<code>_clean_for_speech()</code>

Chapter 23 — `core/audio.py`, and why interrupting is hard

File: 701 lines. You will probably never need to change it.

Read it to understand why interrupting works, then leave it alone unless interrupting stops working.

§23.1 The naive version, and its three failures

"Let the user interrupt" sounds like: if the microphone is loud while Ted is talking, stop talking.

That fails in three separate ways, and all three were found the hard way.

1. A clap is loud. So is a door. Loudness alone is not speech. So there is a voice-activity detector (`webrtcvad`) and a pitch check — human speech has a fundamental frequency of roughly 70–320 Hz, found by autocorrelation. A clap has no pitch. VAD alone calls a clap speech, which is why both are needed.

2. Ted's own voice comes out of the speakers and back into the microphone. So Ted interrupts himself. The native Swift engine uses Apple's Voice Processing to cancel that echo. Without the native engine, use headphones.

3. The big one: Ted was deaf at every sentence boundary. Interrupt detection used to be gated on whether audio was actually playing — and playback goes briefly silent *between sentences*, which is exactly where a human interrupts. So the most natural moment to interrupt was the one moment Ted could not hear you.

The fix was `_in_reply`: keep detection alive across the whole reply rather than each sentence.

That bug is worth remembering as a shape. The condition being tested was *almost* the right one, it was true most of the time, and it was false at precisely the moment that mattered.

§23.2 The sliding window

Several consecutive speech-like frames are required before Ted believes you, so one stray frame cannot cut it off mid-sentence. Roughly a 300 ms window with thresholds for how many frames must look like speech and how many must have pitch.

Every one of those numbers was tuned against real recordings. Changing one blind is how the silent failure comes back.

§23.3 The two modes

`start()` picks one automatically:

Mode	When	Barge-in
<code>aec</code>	the native Swift binary built and started	works over speakers
<code>fallback</code>	pure Python via <code>sounddevice</code>	works on headphones

Which one you got is the first thing to check when barge-in misbehaves. Build the native engine with:

```
cd ~/ted-ai/native && ./build.sh      # needs swiftc: xcode-select --install
```

§23.4 Debugging it

```
TED_DEBUG_BARGE=1 python hud.py
```

That was added as part of the fix, deliberately — because barge-in had died silently once already, with nothing reporting the threshold it was testing against. Making it observable was part of fixing it, not an extra. §34.

Status, honestly: the overhaul was written and unit-tested, and the specific manual test has never been recorded as done — interrupt mid-sentence, interrupt *at* a sentence pause (the case that was broken), confirm typing still interrupts, confirm Ted does not interrupt himself on speakers now that echo cancellation has changed. §35.

PART VI — THE OTHER MODULES

Reference. Do not read this part; look things up in it.

Each entry says what the file owns, the one thing worth knowing about it, and where to change the thing you are most likely to want to change.

Chapter 24 — Your Mac

§24.1 `core/actions.py` — 672 lines

Opening and closing applications, opening a URL in a specific browser, instant Spotify transport, contact lookup, sending an iMessage.

Nearly everything works through **AppleScript** — Apple's built-in scripting language for making one program tell another what to do. Ted writes a short script, hands it to the `osascript` command, and reads what comes back.

The one thing worth knowing: `APPS` is a dictionary mapping the words you say to the real application names macOS expects.

```
APPS = {  
    "vs code": "Visual Studio Code",  
    "news": "News",  
    ...  
}
```

If Ted cannot open something by name, add a line to `APPS`. That is almost always the whole fix.

Every function returns the sentence Ted will say — not a boolean, not a status code. Deliberate, and it is the honesty rule in structural form (§11.8).

§24.2 `core/computer.py` — 345 lines

Typing text, pressing keys, scrolling, reading and writing the clipboard, and the UI inspection tools (`ui_inspect`, `ui_press`, `ui_fill`).

This is macOS **Accessibility** control — the same permission a screen reader needs. It will be silently useless until whatever launches Ted has been granted it in System Settings → Privacy & Security → Accessibility.

Risk note: `type_text` and `clipboard_write` currently run without confirmation. Low risk today because Ted has no browser automation. §35.

§24.3 `core/system_state.py` — 493 lines

A verified picture of what is happening on your Mac right now: which apps are open, what is playing, which browser tab is in front.

The word doing the work is verified. This module reports only what macOS or a media API confirmed *just now*.

The rule it enforces: conversation history is never treated as computer state. An old `"Playing."` tool result from five minutes ago must not let Ted claim music is still playing. What Ted *said* happened and what *is* happening are

two different questions, and only one of them is answered by looking.

`format_for_prompt` turns the snapshot into the short block that goes into the per-turn context, capped at 2,400 characters (§8.3 — it was 900 and that cut off after the app names, defeating the point).

Chapter 25 — Your accounts

§25.1 `core/calendar_app.py` — 191 lines

Real Calendar.app events, through AppleScript. Read and write.

No parallel copy. Ted asks the real app every time. That is the "no second source of truth" principle (§34) — a parallel copy is a copy that will be wrong.

Every function returns `[]` or `""` when AppleScript fails, because AppleScript fails for boring reasons — the app was not running, permission had not been granted — and none of those should stop Ted answering.

§25.2 `core/notes.py` — 126 lines

Apple Notes, through AppleScript. Same shape and same principle as above.

`add_note`, `append_to_note`, `search_notes`, `get_note`.

§25.3 `core/email.py` — 258 lines

Outlook over **IMAP** and **SMTP** — the old, boring, universally supported mail protocols. No OAuth, no browser, no Microsoft sign-in dance.

Set up with `python setup_email.py`.

The honest problem: your password sits in `~/.ted_email_config.json` in plain text. The Microsoft Graph path that would fix this was abandoned about one line from working — MSAL adds `offline_access` to the scopes automatically and rejects the request if you name it yourself. Dropping that one word from the scope list is the fix, and the thing most likely to kill it is admin consent on the school tenant. §35.

§25.4 `core/messages.py` — 298 lines, and `core/bouncer.py` — 210 lines

`messages.py` reads incoming iMessages by opening the Messages database directly. macOS gives no supported way to observe another app's notifications, so this is the only route, and it has two costs stated plainly in the file:

1. It needs **Full Disk Access** granted to whatever launches Ted. Until then every function says so *clearly*, rather than returning empty results. "No new messages" and "I am not allowed to look" are different answers and only one of them is true.
2. The database is opened **read only**, always (`mode=ro`).

`bouncer.py` decides who gets announced. It is a **doorman, not a feed**.

The default posture is **silence**. Getting announced is something a sender earns by being on the list. That default is the whole design: a bouncer that announces everything is a notification centre with extra steps, and the first unknown

short-code that interrupts you mid-lecture is the last day you leave it on.

Rules live in `data/memory.db` , so the dashboard and the running window see the same list.

Chapter 26 — Music

§26.1 `core/spotify_web.py` — 730 lines

The half of music control that needs your account: starting a named playlist, searching for a song, playlist editing. Uses **spotify**, which handles the OAuth token dance and refreshes tokens automatically.

Set up with `python authorize_spotify.py`. Requires Premium.

`_confirm_playing` returns three values — **True, False, or None** — and **None is deliberately not success**. The Web API accepts a play request and returns a success code in plenty of cases where nothing actually starts playing. Saying "Playing X" because the call did not raise is exactly the cheerful-lie failure the honesty rule exists to prevent. §11.8.

§26.2 `core/music.py` — 79 lines

Seventy-nine lines that route a spoken music phrase to one of the two backends.

The split is the interesting part:

Kind	Backend	Why
transport — play, pause, skip	local AppleScript	must be instant
selection — a song, a playlist	Spotify Web API	needs your account

Sending everything through the Web API would make pausing take a network round trip. Sending everything through AppleScript would make "play that Radiohead album" impossible.

Chapter 27 — Seeing

§27.1 `core/attachments.py` — 296 lines

What happens when you drag a file onto Ted, paste a screenshot, or use the attach button.

Images become a data URL on the user message — the same shape `core/screen.py` already sends for screenshots. Qwen handles them on Groq and the local multimodal model handles them offline, so this needed no new provider, only a way in.

Documents and text are extracted to plain text and put in the prompt. Anything long is *also* filed in the knowledge base, so it stays askable after the conversation that introduced it has scrolled away.

Nothing here reaches the network.

One thing worth knowing: attachments belong to exactly one turn. `core/app.py` takes the pending list rather than reading it, and clears it *before* the model call:

```
_attached, self._pending_attachments = self._pending_attachments, []
```

So a file cannot silently ride along on your next message, and a failure mid-turn does not strand it either.

§27.2 `core/screen.py` — 172 lines

Take a screenshot, send it to the model, describe what is on it. Goes through the same provider door as everything else, so it works offline too.

The screenshot is held in memory and never written to disk. Small privacy detail, worth keeping.

§27.3 `core/codebase.py` — 374 lines

Ted reading its own source.

The rule that governs the whole file: Ted can see everything and change nothing without being asked. Reading is free. Writing exists as one narrow function that routes through the same yes/no confirmation as sending a message and refuses to run without it.

Three defences, because "read-only" written in a docstring is not read-only:

1. **Every path is resolved with `realpath` and must land inside the repository.** Resolving *first* and then checking is what stops both `../../.ssh/id_rsa` and a symlink pointing out of the tree. Checking the string before resolving catches neither.
2. **Files git ignores are invisible.** That is what keeps `config.py`, which holds your API keys, out of reach.
3. **Writing requires an explicit confirmation, every time.**

`code_write` takes whole files, not patches. Fine for small modules, wasteful for `core/app.py`. Treat self-editing as suitable for small files only.

Chapter 28 — Doing things later

§28.1 `core/assistant.py` — 446 lines

Reminders, timers, and the parsing that turns "in twenty minutes" into seconds.

Backed by **one JSON file** — `data/assistant.json`. No network, no database, so it survives restarts and cannot take Ted down.

The split it keeps: this module owns the data and the parsing. It never speaks and never touches the window. That separation is what lets it be tested without audio hardware.

§28.2 `core/proactive.py` — 378 lines

Ted bringing things up without being asked: trigger schedules (`daily_at` , `interval_mins` , `weekday_at`) and `daemon_alive()` .

The calendar half has been handed to `ted_daemon.py` . That matters: a thread inside the window dies when you close the window, which is exactly when you most want to be told about your next class.

§28.3 `core/routines.py` — 399 lines

Phrase-to-action routines you write yourself in the dashboard. "Movie mode" closes these apps and sets that volume. Rung 7 of the ladder — zero tokens, no model call.

Matching is deliberately conservative: filler words are ignored, a multi-word phrase may appear inside a longer sentence, and a **one-word phrase must match the entire utterance**. That last rule is what stops a routine named "start" from firing on "start the timer".

Stored in `data/memory.db` so the dashboard and the running window agree.

§28.4 `ted_daemon.py` — 234 lines

The calendar watch, running as a separate program under **launchd** (macOS's service manager) rather than as a thread inside the window.

That is the entire point. See §28.2.

Deliberately narrow. It watches Calendar.app and posts a macOS notification for events starting in the next ~16 minutes. It does *not* fire user-defined triggers, because those can carry actions that only mean something when the assistant is running.

```
venv/bin/python ted_daemon.py --once      # one poll, verbose
bash tools/install_daemon.sh              # install the launchd agent
bash tools/install_daemon.sh --uninstall
tail -f data/ted_daemon.log
```

Status, honestly: the logic is unit-tested; the launchd install has never been verified on macOS. The likely failure is permissions — macOS gates AppleEvents per calling binary, and a launchd-spawned python is a different caller from your terminal, so Calendar access and notifications may both need granting by hand. §35.

Chapter 29 — Plumbing

§29.1 `core/telemetry.py` — 428 lines

Covered at §13.4.

§29.2 `core/remote.py` — 128 lines

A tiny HTTP server so an iPhone Shortcut or a `curl` command can ask Ted something over your local network.

```
GET /ask?token=...&text=...
POST /ask {"text": "..."}
GET /status
```

Security, plainly: set `REMOTE_TOKEN` in `config.py` and every request must carry it. **With no token set, the server does not start at all.** It is never exposed unauthenticated.

That is also the current reason phone access does not work — the token is blank. §35.

§29.3 `core/features.py` — 82 lines

One place that answers "is this optional piece installed and working?"

Every optional subsystem is imported here exactly once, wrapped so a missing dependency disables that feature instead of crashing Ted. Everything else imports the module object and the `HAS_*` flag **from here** rather than re-trying the import.

Why the pattern matters: if ten files each try `import chromadb` in a try/except, you get ten slightly different opinions about whether the knowledge base works. One file, one answer.

§29.4 `core/paths.py` — 26 lines

Where everything lives. Every path is derived from this file's own location rather than hardcoded, so moving or renaming the `ted-ai` folder does not break Ted.

§29.5 `core/logs.py` — 34 lines

The rotating error log. Thirty-four lines, and it exists because of the five-week silent bug.

`print()` is for noise. `error_log` is for things that are actually wrong. Real failures go to `ted_errors.log`, 5 MB × 3 files so the disk never fills.


```
cat ~/ted-ai/ted_errors.log      # real failures only
cat ~/ted-ai/data/ted_launch.log # everything Ted printed
```

The second one matters because launching via `Ted.app` gives you no terminal.

§29.6 `core/intents.py` — 914 lines

Pure text-in, answer-out helpers. No audio, no network, no model, no side effects on import. That is what makes it the most heavily tested file in the project and the safest to change.

Why matching is so forgiving: Whisper almost never returns a clean "stop". It returns "Stop.", "Okay stop", "So, Ted, stop", "Tep.". So every command is matched the same tolerant way — normalise the utterance (drop punctuation and filler), then accept it if it is a command phrase, or if a short utterance *starts with* one. Phrase tables are normalised the same way so the two always line up.

That single choice is why Ted responds to "stop" spoken by a human rather than only to "stop" typed by a programmer.

If Ted does not respond to how you say something, find the `_X_PHRASES` table and add your wording. Then run `python tests/test_intents.py`.

§29.7 `core/hud_bridge.py` — 58 lines

Covered at §12.2.

§29.8 `core/knowledge.py`, `core/lingo.py`, `core/notebook.py`

Chapters 17, 18 and 16 respectively.

PART VII — WORKING ON TED YOURSELF

Chapter 30 — Your first change, step by step

This chapter assumes you have read nothing else. It walks one real, small, useful change from start to finish.

The change: make Ted able to open Discord by saying "open discord".

That is deliberately trivial. The point is the *process* — the commands, the order, and what to check — not the feature.

§30.1 Before you touch anything

```
cd ~/ted-ai
git status                # is anything already half-done?
git log --oneline -5      # what happened recently?
git branch --show-current # should say arch/single-call
```

If `git status` shows files you did not modify, someone else — you, in another session — is mid-task. Do not start on top of that.

Make a branch. This is the single habit that makes everything else safe:

```
git checkout -b add-discord
```

If it goes wrong, `git checkout arch/single-call` and the branch is just gone.

§30.2 Find where it lives

You do not know where app names are handled. Find out:

```
grep -rn "spotify" --include=*.py core/ | head
```

You will see `core/actions.py` come up repeatedly, including a dictionary called `APPS`. Open it:

```
grep -n "APPS = {" -A 20 core/actions.py
```

That is the map from what you say to what macOS expects.

This is the actual skill. Not memorising the codebase — knowing how to find the part of it that matters in about ninety seconds.

§30.3 Make the change

Open `core/actions.py`, find `APPS`, add a line:

```
"discord": "Discord",
```

Keep it alphabetically near its neighbours if they are ordered. Match the formatting around it.

§30.4 Run the tests

```
cd ~/ted-ai && source venv/bin/activate
for t in tests/test_*.py; do printf "%-34s " "$t"; python "$t" | tail -1; done
```

You want every line to say `N passed`, `0 failed`, with the known exceptions in §33.2.

If `test_intents.py` fails, you broke something in `APPS` — a missing comma, a duplicate key. It will name the check.

§30.5 Actually try it

```
python hud.py
```

Type "open discord". Watch the terminal, not just the window.

§30.6 Commit it

```
git add core/actions.py
git commit -m "Ted can open Discord"
```

Write the message for the version of you that reads it in three months. The convention in this repo is a short sentence saying what changed from the user's point of view — read `git log --oneline -20` and you will see the style.

§30.7 If it went wrong

```
git diff          # what did I actually change?
git checkout core/actions.py  # throw away changes to one file
git checkout arch/single-call # abandon the branch entirely
```

Nothing you did on a branch can hurt the working version. That is what branches are for.

§30.8 A slightly bigger second change

Once that works, try one of these. Each is a single-file change with a visible result:

Change	File	Section
Make Ted less formal	<code>SYSTEM_PROMPT</code> in <code>core/llm.py</code>	§9.1
Add wording for a command Ted misses	the <code>_X_PHRASES</code> table in <code>core/intents.py</code>	§29.6
Change the accent colour	<code>:root</code> in <code>ui/ted_hud.html</code>	§19.2
Make Ted keep more conversation history	<code>MAX_HISTORY</code> in <code>core/llm.py</code>	§9.5
Offer a tool the router keeps missing	<code>_FAMILIES</code> in <code>core/routing.py</code>	§7.2

Chapter 31 — Adding a tool, end to end

A tool is how you give Ted a new **capability**. Three files, in order, and the third is the one people forget.

The example: a `flip_coin` tool. Silly on purpose — it keeps the mechanics visible.

§31.1 Step one — the schema

`core/tools.py`, in the `TOOL_SCHEMAS` list:

```
{
  "type": "function",
  "function": {
    "name": "flip_coin",
    "description": (
      "Flip a coin and report heads or tails. Use when the user asks to "
      "flip a coin, or asks you to decide between exactly two options at "
      "random. Do not use for a considered choice – say what you think "
      "instead."
    ),
    "parameters": {
      "type": "object",
      "properties": {
        "count": {"type": "integer",
                  "description": "How many coins, 1-10. Default 1."}
      },
      "required": []
    }
  }
},
```

Two things about that description:

- It says **when** to use it, not what it does.
- It says **what to do instead** when it does not apply. Half of a good description is fencing it off from its neighbours.

`count` is not in `required`, so it may be absent from the arguments entirely.

§31.2 Step two — the work

Small enough to go straight into the dispatch branch. Something bigger belongs in `core/tool_handlers.py` or its own module.

§31.3 Step three — the branch (the one people forget)

`core/app.py`, in `_dispatch_tool` — search for [BOOK §11.6] :

```
if name == "flip_coin":
    import random
    n = max(1, min(10, int(args.get("count", 1) or 1)))
    flips = [random.choice(("heads", "tails")) for _ in range(n)]
    return "Flipped: " + ", ".join(flips) + "."
```

Note `args.get("count", 1)`, not `args["count"]`. Note the clamp — the model can and will send 500 eventually.

Without this branch, the model calls `flip_coin`, nothing happens, and the model cheerfully narrates a result it invented. The failure is invisible from the outside. This is the step.

§31.4 Step four — routing

`core/routing.py`, `_FAMILIES`, so the tool is actually offered:

```
(r"\b(?:flip|coin|heads or tails|toss)\b", ("flip_coin",)),
```

Without this the tool only appears after a `find_tools` round trip. It will still work — that is the escape hatch doing its job (§7.3) — but it costs an extra call every time.

§31.5 Step five — is it an action?

Does it change the world? A coin flip does not, so leave `ACTION_TOOLS` alone.

If yours *does* — sends something, writes something, opens something — add its name to `ACTION_TOOLS` in `core/tool_handlers.py`, or the honesty check cannot tell whether anything happened (§11.8).

If it is **destructive**, add it to `needs_confirmation` too (§11.7).

§31.6 Step six — test it

```
# tests/test_flip.py
import os, sys
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from core.tools import TOOL_SCHEMAS

names = [s["function"]["name"] for s in TOOL_SCHEMAS]
assert "flip_coin" in names, "schema missing"

src = open("core/app.py").read()
assert 'if name == "flip_coin"' in src, "no dispatch branch"
print("2 passed, 0 failed")
```

That second check looks crude and it is exactly right. It catches the one mistake that is otherwise invisible. Several real test files in this repo do the same thing.

§31.7 The checklist

Step	File	Skip it and...
schema	<code>core/tools.py</code>	the model does not know it exists
handler	wherever the work lives	nothing to call
dispatch branch	<code>core/app.py</code>	silently does nothing
routing family	<code>core/routing.py</code>	costs a <code>find_tools</code> round trip
<code>ACTION_TOOLS</code>	<code>core/tool_handlers.py</code>	honesty check goes blind
<code>needs_confirmation</code>	<code>core/tool_handlers.py</code>	destructive things run unasked
a test	<code>tests/</code>	you find out in three weeks

§31.8 The cost, stated

Every schema is text sent on every turn that selects it. The whole catalogue was measured at ~3,645 tokens against a 6,000-per-minute ceiling.

Adding a tool is not free. Keep descriptions tight, and if you add several, look at the `tools=` figure in the `[prompt]` log line to see what it cost.

Chapter 32 — How to read an error

§32.1 The two logs

```
cat ~/ted-ai/ted_errors.log      # real failures only
cat ~/ted-ai/data/ted_launch.log # everything Ted printed
```

The second one matters because launching via `Ted.app` gives you no terminal at all. If Ted misbehaved an hour ago and you were not watching, that file is where it happened.

§32.2 Reading a Python traceback

Python prints the deepest call **last**. Read from the bottom up.

```
Traceback (most recent call last):
  File "core/app.py", line 2145, in _dispatch_tool
    topic = news.add_topic(args["label"])
KeyError: 'label'
```

Bottom line: **what went wrong**. `KeyError: 'label'` — something asked a dictionary for a key that was not there.

Line above: **where**. `core/app.py`, line 2145.

The rest: how it got there.

Common ones in this codebase:

Error	Almost always means
<code>KeyError: 'x'</code>	<code>args["x"]</code> on an optional argument. Use <code>.get()</code> .
<code>AttributeError: 'NoneType' has no attribute 'y'</code>	something returned <code>None</code> and you did not check
<code>TypeError: ... takes 2 positional arguments but 3 were given</code>	forgot <code>self</code> , or an argument count changed
<code>ModuleNotFoundError</code>	an optional dependency, or the venv is not active
<code>sqlite3.OperationalError: no such column</code>	you changed the schema and skipped <code>_migrate</code> (§14.7)

§32.3 The tags to grep for

Ted tags its log lines. These are the useful ones:


```
cd ~/ted-ai
grep '\[provider\]' data/ted_launch.log # a fall back to the local brain
grep '\[honesty\]' data/ted_launch.log # Ted claimed an action it did not take
grep '\[timing\]' data/ted_launch.log # where the wait went
grep '\[prompt\]' data/ted_launch.log # how big each prompt was
grep '\[tools\]' data/ted_launch.log # which tool was called, with arguments
grep '\[memory\]' data/ted_launch.log # what was remembered, or deliberately not
```

`[timing]` deserves a note. It used to measure first token from *inside* the stream — after the request had already been accepted — so every wait that mattered happened outside the only number being printed. There are now `request accepted after Nms` and `turn to first output Nms`, and the gap between them is where the truth lives.

§32.4 When there is no error at all

The harder case, and the one this codebase has been bitten by repeatedly.

Ted did nothing and said it did something. Check `[honesty]`. Then check that the tool has a dispatch branch (§31.3).

Ted is slow with nothing in the log. Check `[timing]`, then `[provider]` for a rate limit, then whether Ollama is being waited on.

A feature silently does not work. Check `core/features.py` — an optional dependency may be missing and the feature quietly disabled.

The window does nothing when you click. A `tedHud` function Python is calling does not exist, and `js()` swallowed the error (§12.2). Or an element id typo. §19.4.

§32.5 Two escape hatches

```
TED_DEBUG_BARGE=1 python hud.py # verbose barge-in decisions
TED_LEGACY_LADDER=1 python hud.py # the old two-call path, for bisecting only
```

The second is not a supported mode. It exists so you can answer "did this regression come from the single-call change?" without guessing.

Chapter 33 — The test suite

§33.1 Running it

```
cd ~/ted-ai && source venv/bin/activate
for t in tests/test_*.py; do printf '%-34s ' "$t"; python "$t" | tail -1; done
```

32 files, roughly 1,150 checks. Each prints its own summary.

There is **no test framework**. No pytest, no unittest. Each file is a script with a `check(description, condition)` helper that counts passes and failures and exits non-zero if anything failed.

That is unusual and it is a good fit here: you can run one file directly, the output is readable without tooling, and adding a test means adding a line.

§33.2 The three known failures

These fail and are not your fault:

File	Check	Why
<code>test_bouncer.py</code>	"a blocked read explains how to fix it"	needs Full Disk Access
<code>test_codebase.py</code>	two git-history checks	only pass where there is a real <code>.git</code>

Anything else failing is something you did.

§33.3 The one to be careful about

`tests/test_pipeline.py` — 122 checks — is a **characterization test**. It pins the ladder's interception order, the tool loop, the compose and disambiguation flows, mute, and both history trims.

It documents what Ted *currently does*, including two quirks pinned deliberately **as-is** rather than fixed:

- "the second one" matches the ordinal "one" and picks the **first** candidate
- "nevermind" during contact disambiguation had a swallowing bug that has since been addressed in the cancel branch — worth re-checking whether the pin still reflects reality

If you change the ladder and this file fails, you have changed behaviour. That might be exactly what you wanted. Update the test *deliberately*, and say so in the commit message.

§33.4 Writing one

Copy the top of any existing file:

```

import os, sys
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from core import intents

PASS = FAIL = 0
def check(desc, cond):
    global PASS, FAIL
    if cond: PASS += 1; print(f" ✓ {desc}")
    else:    FAIL += 1; print(f" x {desc}")

check("'shut up' stops Ted", intents._is_stop_command("shut up"))

print(f"\n{PASS} passed, {FAIL} failed")
sys.exit(1 if FAIL else 0)

```

Write the description as a sentence about behaviour, not about code. `"'shut up' stops Ted"` tells you what broke. `"test_stop_2"` does not.

§33.5 What the tests cannot tell you

Everything in `tests/` runs pure Python. It can be run off the Mac entirely, by stubbing the `groq` module — that is how these suites get checked from a Linux sandbox.

It proves logic and proves nothing about macOS.

No test can tell you that AppleScript actually opened Calendar, that the microphone permission was granted, that Kokoro produced audible sound, that the native audio engine started, or that a Groq → Ollama handover completed in a tolerable time.

Those need running Ted on the Mac and watching. Any claim about them that has not been watched should be labelled unverified — which is a habit worth keeping in your own notes, because you have been burned by exactly this.

Chapter 34 — The rules this codebase lives by

Eleven principles that recur throughout Ted's history. Each earned its place by being violated first.

1. Math in Python, words in the model

A language model doing "8 percent of 250" fails **silently**. A wrong number reads exactly like a right one, there is nothing to log and nothing to notice. Arithmetic stays in `_parse_calc`. The model narrates; Python computes.

2. Silent failures are the expensive ones

Fact extraction was dead for five weeks because an exception died inside a `print()`. Barge-in died silently because nothing reported the threshold it was testing.

Both fixes included **making the thing observable**. That was not extra work tacked on; it was part of the fix. Real failures go to `ted_errors.log`, not stdout.

3. Ground truth over optimism

Action tools report what actually happened, and Ted speaks that verbatim. `_confirm_playing` returns three values because two would force a lie. §11.8.

4. Cheap gates before expensive ones

The ladder is sound. What was once wrong was how many rungs were hardcoded, and that two rungs did the same expensive work twice. Both fixed. The principle survived the fix.

5. Deleting a regex is a feature

Every pattern removed moves a decision from *hardcoded* to *reasoned*, and makes Ted feel more intelligent. Keyword-gated tool triggering was removed because it "made Ted feel like a robot spitting back answers" and locked novel phrasings out of every tool. Any proposal to add keyword matching back needs a strong reason.

6. No second source of truth

Calendar and Notes go through AppleScript to the **real apps** rather than keeping a parallel copy. A parallel copy is a copy that will be wrong.

7. Selective memory beats complete memory

Remembering everything makes callbacks worse. A memory list full of "Charlie set a two minute timer" is worse than no memories at all. §13.3.

8. Prompt prefix stability is a performance feature

Keep the static prefix byte-identical; put volatile per-turn context last. `stable_window`, the message order, the two tool-guidance shapes — all the same idea. §9.4, §9.5.

9. Know which tool can run the thing

You work on Ted from two places, and they can do different things:

	Cowork (Linux sandbox)	Claude Code (on the Mac)
read and edit the repo	yes	yes
run the pure-Python tests	yes, with <code>groq</code> stubbed	yes
run Ted	no	yes
AppleScript, CoreAudio, the venv	no	yes
call Groq with the real key, reach Ollama	no	yes

Runtime, audio, hardware and permission bugs belong on the Mac. Diagnose in one, hand off to the other, and **state plainly which claims are unverified**.

10. Two places must never own one fact

The recurring bug in this codebase is not complexity, it is **duplicated judgment**:

- a gate matching by substring while the dispatch it guarded matched by prefix
- a stored reply diverging from the spoken one
- a health check inferring "cloud down" from "no calls yet"
- four retrieval timeouts each spending the whole budget

Every one of those was two pieces of code answering the same question differently. **When you add a check, find who already answers it**.

11. Plans rot faster than code

An audit found that roughly a third of a standing feature list's "already built" items were wrong within seven weeks, and two "planned" items had quietly shipped.

Before planning off any document — including this one — re-read the repo:

```
cd ~/ted-ai
git status && git log --oneline -10
python tools/ted_map.py --markdown      # generated FROM the code
sqlite3 data/memory.db ".tables"
```

`tools/ted_map.py` reads the code. This book was written by hand. **Where they disagree, the script is right.**

Chapter 35 — What is already known to be wrong

Check this list before spending an afternoon rediscovering something.

§35.1 `core/app.py` is nearly 4,000 lines

The decomposition was scoped in June. Characterization tests — stage one — landed in August. **No code has moved.** The file was ~103 KB when scoped and is larger now. It is growing faster than it is being cleaned.

The planned first step: the old `_assistant_command` dispatch chain is the seam. Its branches can each move into the module they already delegate to — email to `email.py`, reminders to `assistant.py`, music to `music.py`. `test_pipeline.py` is the safety net. **One domain per commit, green before the next.**

This has been repeatedly deferred in favour of features. That is a real decision you keep making, not an oversight — but it should be a decision you are making knowingly.

§35.2 The rate-limit ceiling

Roughly 6,400 tokens per message against a 6,000-tokens-per-minute free tier. One message a minute fits; a conversation is by definition several in a row.

Two ways out: cut prompt weight (the `[prompt]` and `ctx_breakdown` numbers tell you where it is going), or pay for a tier without the ceiling. Removing the news watcher and voice ID clawed back four tool schemas, which helps and does not solve it.

§35.3 Things built but never verified on the Mac

Each of these is written, unit-tested, and **has never been observed running on real hardware**. They should not be described as working.

Thing	The specific unverified claim	Section
The Groq → Ollama handover	that it completes in tolerable time rather than reading as a hang	§10.4
The calendar daemon	that launchd can grant it Calendar access at all	§28.4
Barge-in since the fix	the sentence-boundary case in particular	§23.4

§35.4 Security items

Live API keys are in a Google Doc in plain text — Groq, ElevenLabs, and Airtable tokens including production ones for another project. One bad share link from public. Rotate them and move them to a password manager. Ten minutes, and the only item here with an unbounded downside.

The Outlook password is cleartext in `~/ted_email_config.json`. §25.3.

`type_text`, `clipboard_write` and app control run unconfirmed. §11.7.

§35.5 Dead weight

Thing	State
<code>patterns</code> table	rows accumulating, nothing reads them
<code>habit_logs</code> table	built, never used
<code>goals</code> table	leftover from a deleted feature
<code>news_topics</code> / <code>news_items</code> tables	left behind by the news removal, nothing reads them
<code>TED_REFERENCE.txt</code>	last updated June; describes three things that no longer exist. Actively misleading.

§35.6 Two pinned quirks

Documented in the test suite as *current behaviour*, not as correct behaviour:

- "the second one" matches the ordinal "one" and picks the **first** candidate
- "nevermind" during contact disambiguation — the cancel branch now clears pending state, so re-check whether the pin still reflects reality

Fixing either means updating `test_pipeline.py` deliberately.

§35.7 The open question

After a day of real use in August: *"I cannot get a conversation out of ted. he freezes every damn line."*

Six mechanical causes were found and fixed — the busy lock, the SDK's silent retries, four retrieval timeouts each spending the whole budget, a dead Ollama costing 23 seconds a message, and two lies of the intent-reported-as-outcome shape.

Whether that was enough has not been established. The three numbers that settle it, after one ordinary conversation:

```
cd ~/ted-ai
grep -c '\[provider\]' data/ted_launch.log          # rate limits
grep '\[honesty\]' data/ted_launch.log             # phantom actions
grep '\[timing\] turn to first output' data/ted_launch.log
```

Settle it with logs, not argument. Everything else on any roadmap is guesswork until that is known.

APPENDIX A — Every file, one line each

Line counts as of August 19, 2026, after the commenting pass.

The entry points

File	Lines	What it owns
<code>hud.py</code>	345	Starts everything. Window, dashboard thread, teardown. Ch. 4
<code>ted_daemon.py</code>	234	The calendar watch, outside the window process. §28.4
<code>config.py</code>	—	Your keys and settings. Not in git.
<code>config.example.py</code>	—	The template for the above.

The message path

File	Lines	What it owns
<code>core/app.py</code>	3,973	The ladder, the tool switchboard, the JS API, background threads. Ch. 5, 6, 11
<code>core/routing.py</code>	743	Tool menu selection, reflexes, cloud-vs-local. Ch. 7
<code>core/llm.py</code>	2,055	Persona, prompt assembly, the streamed turn, facts, summaries. Ch. 8, 9, 11, 13
<code>core/providers.py</code>	975	The one door. Groq, Ollama, the handover, the pin. Ch. 10
<code>core/tools.py</code>	1,329	The tool menu the model reads. §11.4
<code>core/tool_handlers.py</code>	886	What several tools do; the honesty and confirmation rules. §11.5–11.8
<code>core/intents.py</code>	914	Pure text helpers. Heavily tested. §29.6
<code>core/hud_bridge.py</code>	58	Python → JavaScript. §12.2

Memory

File	Lines	What it owns
<code>core/memory.py</code>	874	The SQLite database. Facts, exchanges, sessions. Ch. 14
<code>core/notebook.py</code>	423	Named pages of numbered entries. Ch. 16
<code>core/knowledge.py</code>	325	ChromaDB vector store for documents. Ch. 17
<code>core/lingo.py</code>	206	Your shorthand, expanded before routing. Ch. 18
<code>dashboard/app.py</code>	691	The Flask server behind the panels. §15.1–15.2
<code>dashboard/db.py</code>	616	Schema, table registry, the audit triggers. §15.3–15.4

Voice and audio

File	Lines	What it owns
<code>core/voice.py</code>	728	Ears and mouth. Whisper, Kokoro, the capture gates. Ch. 22
<code>core/audio.py</code>	701	Raw audio, barge-in, the two engine modes. Ch. 23
<code>native/ted_audio.swift</code>	—	The full-duplex audio engine with echo cancellation.

Your Mac and your accounts

File	Lines	What it owns
<code>core/actions.py</code>	672	Apps, browsers, transport, contacts, iMessage. <code>APPS</code> . §24.1
<code>core/computer.py</code>	345	Typing, keys, clipboard, UI inspection. §24.2
<code>core/system_state.py</code>	493	Verified snapshot of what is happening now. §24.3
<code>core/calendar_app.py</code>	191	Calendar.app via AppleScript. §25.1
<code>core/notes.py</code>	126	Apple Notes via AppleScript. §25.2
<code>core/email.py</code>	258	Outlook over IMAP/SMTP. §25.3
<code>core/messages.py</code>	298	Reading the Messages database, read-only. §25.4
<code>core/bouncer.py</code>	210	Who gets announced. Silence by default. §25.4
<code>core/spotify_web.py</code>	730	Playlists and search, via your account. §26.1
<code>core/music.py</code>	79	Routing between the two music backends. §26.2

Seeing, and doing things later

File	Lines	What it owns
<code>core/attachments.py</code>	296	Dropped files → what the model receives. §27.1
<code>core/screen.py</code>	172	Screenshot and vision. §27.2
<code>core/codebase.py</code>	374	Ted reading its own source, safely. §27.3
<code>core/assistant.py</code>	446	Reminders, timers, duration parsing. §28.1
<code>core/proactive.py</code>	378	Trigger schedules, daemon liveness. §28.2
<code>core/routines.py</code>	399	Phrase → actions, zero tokens. §28.3

Plumbing

File	Lines	What it owns
<code>core/telemetry.py</code>	428	One row per turn. §13.4
<code>core/remote.py</code>	128	The HTTP endpoint for Shortcuts. §29.2
<code>core/features.py</code>	82	One answer to "is this optional piece working?". §29.3
<code>core/paths.py</code>	26	Where everything lives. §29.4
<code>core/logs.py</code>	34	The rotating error log. §29.5

The window

File	Lines	What it owns
<code>ui/ted_hud.html</code>	2,068	The entire window. Ch. 19–21
<code>dashboard/index.html</code>	800	The Memory page
<code>dashboard/notebook.html</code>	280	The Notebook page
<code>dashboard/diagnostics.html</code>	311	The per-turn Diagnostics page
<code>ui/ted_hud_orb.html</code>	—	The old voice-orb variant, kept

Tools

File	What it does
<code>tools/ted_map.py</code>	Generates a status block from the code . Trust it over documents.
<code>tools/make_app.sh</code>	Builds <code>Ted.app</code>
<code>tools/make_icon.py</code>	Pure-stdlib PNG icon generator
<code>tools/install_daemon.sh</code>	Installs the launchd agent
<code>tools/install_hooks.sh</code>	The pre-commit hook that runs <code>ted_map.py --sync</code>
<code>native/build.sh</code>	Builds the Swift audio engine

APPENDIX B — The anchor index

Every `[BOOK §...]` tag currently in the code, and where it is.

Anchor	File	What it marks
§5.1	core/app.py	ask() — the typed way in
§5.2	core/app.py	conversation_loop() — the spoken way in
§6	core/app.py	_respond() — the ladder, with the full rung list
§6.2	core/app.py	rungs 1–4: mute, stop, cancel, UI
§6.3	core/app.py	rung 5: pending flows
§6.4	core/app.py	rung 6: lingo
§6.5	core/app.py	rungs 7–9: routines, documents, reflexes
§6.6	core/app.py	_use_deterministic_command() — gate 5
§6.7	core/app.py	rung 11: the model
§7.2	core/routing.py	select_tool_schemas()
§7.4	core/routing.py	plan_reflex()
§7.5	core/routing.py	classify_brain()
§8 , §9 , §11.1–11.3	core/llm.py	ask_streaming()
§9.1	core/llm.py	SYSTEM_PROMPT
§9.5	core/llm.py	stable_window()
§10.1	core/providers.py	chat_create() — the door
§10.3	core/providers.py	_ensure_ollama()
§10.6	core/providers.py	get_provider_mode() — the pin
§11.2	core/llm.py	ToolRuntime
§11.4	core/tools.py	TOOL_SCHEMAS
§11.6	core/app.py	_dispatch_tool() — the switchboard
§11.7	core/tool_handlers.py	needs_confirmation()
§11.8	core/tool_handlers.py	ACTION_TOOLS , looks_like_failure()
§11.8	core/llm.py	claims_completed_action()
§13.2	core/llm.py	extract_and_save_facts()
§13.3	core/llm.py	generate_session_summary()
§14.2	core/memory.py	save_fact()
§14.5	core/memory.py	memory_event()
§16.3	core/notebook.py	index_line()

Anchor	File	What it marks
<code>§22.3</code>	<code>core/voice.py</code>	<code>capture()</code> — the gates

To find any of them:

```
cd ~/ted-ai
grep -rn "BOOK §11.6" .
```

Every source file also opens with a `READING THIS FILE` block naming its chapter. To list them all:

```
grep -rn "READING THIS FILE" --include=*.py --include=*.html . | head -50
```

APPENDIX C — Glossary

AEC — acoustic echo cancellation. Stopping Ted hearing its own voice through the speakers.

API — application programming interface. A defined way for one program to ask another program to do something.

AppleScript — Apple's scripting language for making one Mac program tell another what to do. Ted runs it via the `osascript` command.

barge-in — interrupting Ted by talking over it. Ch. 23.

ChromaDB — the vector database behind the knowledge base. Ch. 17.

daemon thread — a background thread that does not keep the program alive. Every watcher in Ted is one. §2.6.

embedding — a list of numbers representing roughly what a piece of text means, so that similar meanings end up numerically close. §17.1.

Flask — the Python library the dashboard is written in. §15.1.

FTS5 — SQLite's built-in full-text search. §14.3.

generator — a function containing `yield` that hands back values one at a time and pauses in between. How streaming works. §2.7.

Groq — the hosted service that runs the model. Free tier, with a tokens-per-minute ceiling.

IMAP / SMTP — the standard protocols for reading and sending email. §25.3.

JSON — JavaScript Object Notation. A text format for structured data. Tool arguments arrive as JSON.

Kokoro — the local text-to-speech model. Voice `am_michael`. §22.2.

launchd — macOS's service manager. Runs `ted_daemon.py`. §28.4.

Ollama — the program that runs models locally on your Mac. §10.3.

ONNX — a portable format for running machine-learning models without a heavy framework. Kokoro and fastembed both use it.

prompt caching — a provider skipping reprocessing of a prompt prefix it has seen before, while that prefix is byte-identical. §9.4.

pywebview — the library that puts a browser engine in a Mac window and bridges it to Python. §4.2.

regex — regular expression. A pattern for matching text. §2.9.

schema — one tool's description: name, when to use it, arguments. §11.4.

SQLite — a database that is a single file rather than a server. §14.1.

STT / TTS — speech to text / text to speech.

token — roughly three-quarters of a word. What providers count and charge for.

tool call — the model asking Ted to run something, by name, with arguments. Ch. 11.

venv — virtual environment. An isolated set of Python packages for one project. `source venv/bin/activate`.

WAL — write-ahead logging. The SQLite mode that lets many threads read while one writes. §14.8.

Whisper — the speech-to-text model. Hosted on Groq by default, local as a fallback. §22.2.

Colophon

Written August 19, 2026, from the working tree of `~/ted-ai` on branch `arch/single-call`, after the removal of the pet, the news watcher, and voice ID.

Every file in the repository carries a `READING THIS FILE` block pointing at its chapter, and every significant section carries a `[BOOK §...]` anchor.

The numbers in this book were true on the day it was written. The anchors are maintained by the code itself. Where the two disagree, trust the anchors — and trust `python tools/ted_map.py --markdown` over both, because it reads the code rather than remembering it.